

Scry 技术架构 与 Agent 实现文档

Scry 技术团队

文档版本: 1.0.0 适用架构: Scry Production Architecture

适用范围: 平台服务、智能运维 Agent、Loop 自动化与安全控制

Abstract—Scry 是面向研发、运维和平台工程团队的一体化可观测与智能运维系统。平台以 OpenTelemetry Collector 接收指标、日志和链路数据,以 ClickHouse 承担遥测分析存储,以 Go Query Service 提供统一查询、告警、仪表盘、组织权限和平台配置能力,并通过 React 前端形成从服务总览、链路追踪、日志检索、基础设施监控到故障处置的完整工作台。系统的核心特色是把可观测数据、证据治理、运行记忆和受控执行组成同一条 Agent 闭环:TopoMem 使用当前事故 Trace 的异常调用路径跨越服务词汇差异,在根因服务区域内按错误签名和记忆分块重排;证据链记录每次工具结果的来源、置信度、关联度与人工处置;MCP 工具协议把自然语言意图编译为确定性查询;Loop 则把诊断、安全决策、审批、执行和验证固化可为恢复流程。Scry Agent Service 基于 Mastra、AI SDK、Hono、MCP、PostgreSQL 与 WASM SQLite 实现模型适配、持久对话、TopoMem 检索、证据验证写回、A2UI 解释、Prompt Injection 防护、最小权限 SSH 和记忆自维护。本文按正式生产架构说明系统分层、完整功能、Agent 与 TopoMem 实现、数据模型、接口契约、安全控制、部署和运维流程,为开发、交付、实施和安全评审提供统一技术依据。

Index Terms—可观测性, 智能运维, Agent, TopoMem, 运行拓扑记忆, 证据链, MCP, Loop, 人工审批, RBAC

I. 引言

A. 文档定位

本文是 Scry 的正式技术架构文档,描述生产版本的组件职责、服务边界、运行协议、数据结构和运维方式。文档同时承担四类用途:开发团队据此理解模块依赖和扩展点;平台团队据此完成部署、容量规划与日常运维;实施团队据此配置数据接入、模型、Loop 和 SSH 策略;安全团队据此审查身份继承、秘密管理、人工审批与审计闭环。

本文覆盖 Scry Web Console、Query Service、Agent Service、OpenTelemetry Collector、ClickHouse、Alertmanager、ZooKeeper、Schema Migrator 以及受管 SSH 主机。外部模型服务被视为可替换依赖,通过 OpenAI Responses 或 Anthropic Messages 协议接入。所有用户可见功能、后台调度、事件触发、审批恢复和审计链路均按正式交付状态描述。

B. 适用读者

- **后端与前端开发者**: 关注接口、模块、状态流、数据模型和错误语义。
- **Agent 与自动化开发者**: 关注模型配置、记忆、工具、A2UI、Loop、调度器和审批机制。
- **SRE 与平台工程师**: 关注端口、依赖、健康检查、容量、备份、高可用和故障恢复。
- **实施与解决方案团队**: 关注功能边界、数据接入、权限角色、 workflows 配置和主机纳管。

- **安全与合规人员**: 关注 JWT、RBAC、密钥加密、SSH 主机指纹、命令白名单和审计日志。

C. 系统边界与术语

Scry 将系统拆分为遥测数据面、平台控制面、智能执行面和交互呈现面。遥测数据面负责接收和存储 metrics、logs 与 traces;平台控制面负责查询、规则、仪表盘、组织和访问控制;智能执行面负责模型推理、工具、Loop、调度与审批;交互呈现面负责 Web 页面、实时流、A2UI 和运维操作。

本文使用以下术语: **Thread** 表示一个持久化 Agent 任务对话; **Run** 表示一次 Agent 或 Loop 执行; **Step** 表示模型步骤、工具调用或 Loop 节点; **Tool** 表示具备严格输入输出 Schema 的受控能力; **Loop** 表示可版本化、可调度、可暂停恢复的自动化流程; **A2UI** 表示由工具返回、前端按协议渲染的结构化界面数据; **TopoMem** 表示使用事故时运行拓扑约束候选服务区域、再以错误签名和内容相似度重排的运维记忆检索机制; **证据** 表示由遥测、平台对象、文档、Loop 或受控主机工具产生并具有来源与生命周期的事实材料。

II. 核心技术特色与运行主线

A. 差异化技术主线

Scry 不是在观测平台旁附加一个通用聊天入口,而是把 Agent 的每一步限制在可验证、可解释、可恢复的系统路径内。完整观测功能提供实时事实基础,TopoMem 提供跨事故复用的运维经验,证据链决定哪些材料可以进入结论和长期记忆,MCP 负责把模型意图收敛为受约束工具参数,Loop 负责把一次性诊断提升为可重复执行的组织流程。表 I 按重要性列出系统主线。

B. 证据、记忆与执行闭环

图 1 给出 Scry 的主要运行闭环。TopoMem 位于“获得事故上下文”与“形成首个诊断假设”之间,作用不是替代当前遥测,而是利用已验证历史经验选择首个服务目标和工具方向。任何记忆命中都以不可信历史数据进入 Prompt,必须由本轮工具证据重新验证;只有被接受的证据或具有可用证据支撑的 Loop 结果才能写回长期记忆。该约束把论文提出的运行拓扑检索思想[1]落实为带权限、证据、维护和审计边界的产品路径。

闭环中的主次关系是明确的:当前遥测证据优先于历史记录,服务端策略优先于模型工具意图,人工证据处置优先于自动评分,验证节点优先于操作节点的自述结果。TopoMem 提高首个目标选择质量,但不会把记忆直接提升为事故事实;Loop 提高执行复用程度,但不会绕过角色、审批或命令白名单。

TABLE I
Scry 核心技术特色及其系统作用

技术主线	解决的问题	实现机制	可验证输出
TopoMem 运行拓扑记忆	症状服务与根因服务词汇不一致，平面检索把 Agent 引向错误服务	从当前证据提取 Trace 边，枚举异常路径和多分支候选，在服务局部执行签名、词项、稠密与质量重排；无可靠路径时选择性回退	检索策略、症状服务、匹配路径、签名词、分块得分、回退原因和检索运行 ID
证据治理闭环	模型结论缺少来源，历史经验与当前事实混用	工具结果自动入证据库；计算置信度和关联度；支持接受、驳回、挂起、备注、选择和基于更新证据重答	证据 UUID、来源、状态、评分因子、关系、操作历史及回答修订记录
确定性 MCP 工具协议	模型猜测底层 Query API、字段包装或资源 ID，造成不可控试探	自描述 JSON Schema、扁平高层参数、服务端校验、JWT 权限继承、结果预算和 A2UI 映射	toolCallId、规范化输入、结构化输出、错误分类、耗时和审计事件
安全 Loop 执行	复杂运维任务无法稳定重复，自动操作缺少审批、恢复和验证	顺序、条件、并行、循环、MCP、安全检查、审批、受控操作和验证节点统一编译为持久 Run	Workflow Version、步骤状态、检查点、审批快照、运行事件、结果与重试记录
统一可观测数据基础	Agent 缺少可信实时事实或绕过平台权限直接访问存储	OpenTelemetry、ClickHouse 与 Query Service 统一指标、日志、链路、告警、仪表盘和基础设施查询	有界时间窗、服务和属性过滤、查询状态、原始记录、聚合结果与 Trace 关联

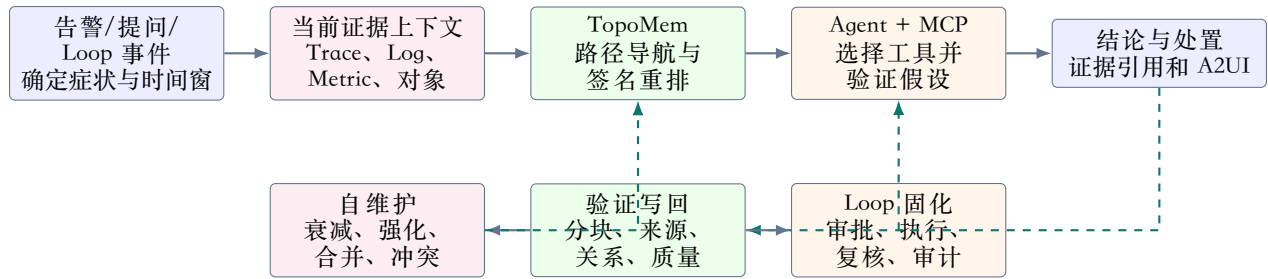


Fig. 1. Scry 证据、TopoMem、Agent 与 Loop 闭环。实线表示本轮处理，虚线表示验证后的经验回流。

III. 总体架构

A. 设计目标

Scry 的架构设计遵循以下原则。

- **统一观测**：指标、日志、链路和异常共用查询与可视化入口，减少多套工具之间的上下文切换。
- **自托管数据面**：遥测数据保存在 ClickHouse，平台配置保存在 SQLite，Agent 记忆和运行状态保存在 PostgreSQL，主机端口默认绑定回环地址。
- **权限继承**：Agent 不直连 ClickHouse，而是携带当前用户 JWT 调用 Query Service，使原有 RBAC 继续生效。
- **证据驱动诊断**：Agent 指令要求先查询、后结论，并区分事实、推断和行动项；复杂诊断先发布计划。
- **拓扑引导记忆**：历史经验先按事故时异常调用路径定位服务区域，再按错误签名和分块内容排序；缺少可靠 Trace 时显式回退，避免伪造拓扑确定性。
- **验证后学习**：记忆写回必须绑定可用证据或 Loop 运行结果；已驳回、挂起和注入风险内容不能进入在线长期记忆。
- **受控操作**：工具参数由 Zod 约束，查询和输出设置上限，SSH 执行受主机、命令、超时和审批策略共同控制。
- **状态可恢复**：对话、工作流定义、运行事件、步骤状态和审批点持久化，服务重启后可以继续读取和恢

复。

- **自动化可治理**：Loop 同时支持手动、Cron 和事件触发，所有入口都进入统一的运行记录、幂等和审计链路。
- **协议可替换**：模型端点、遥测协议和呈现协议通过适配层隔离，避免业务逻辑绑定单一供应商。

B. 分层视图

图 2 给出系统分层。OpenTelemetry 定义厂商中立的遥测数据模型与传输规范[2]，ClickHouse 提供面向分析负载的列式存储[3]。Scry 在数据接入和分析存储之上构建统一 Query Service，并以独立 Agent Service 承载智能执行、自动化调度和高风险操作控制。

C. 服务职责

D. 数据面与控制面

系统存在四条相互隔离但可关联的数据路径。遥测写入路径从被观测应用经 OTLP 进入 Collector，再写入 ClickHouse；交互查询路径从浏览器或 Agent 进入 Query Service，由查询构建器生成 ClickHouse 查询并规范化返回；平台控制路径保存用户、组织、告警、仪表盘、集成和偏好；智能执行路径保存模型设置、Thread、UI Message、证据、TopoMem、Loop、Run、SSH 主机、策略和审计。将 Agent 放在 Query Service 之外而不是放

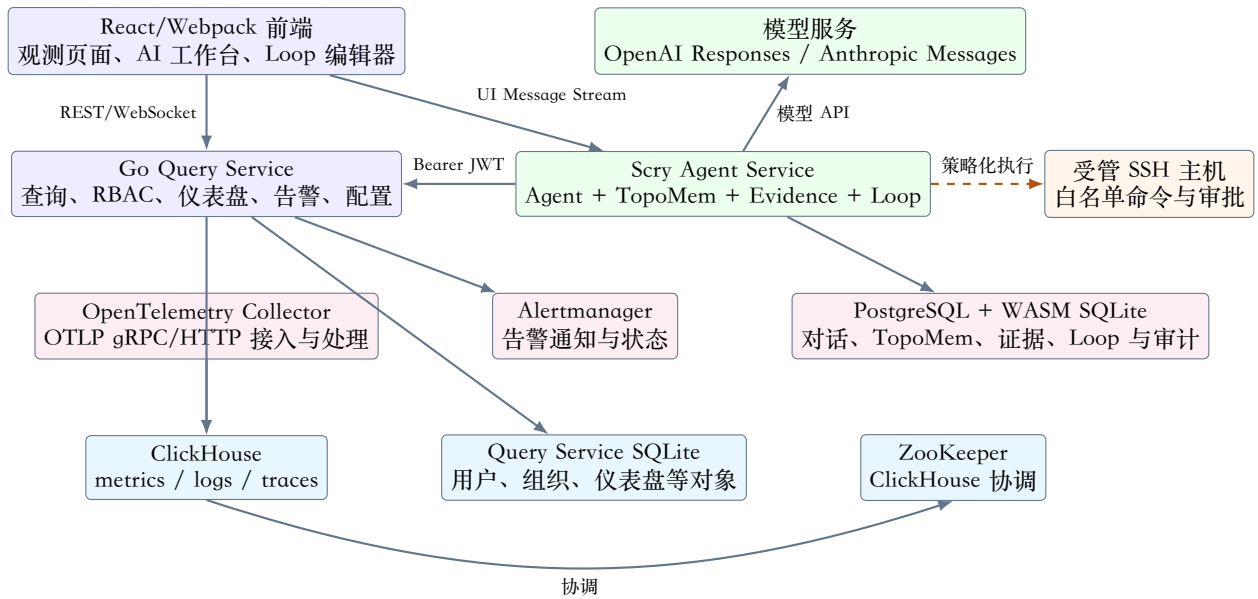


Fig. 2. Scry 总体架构与信任边界。虚线表示必须经过策略和人工审批的 SSH 路径。

TABLE II
运行服务及职责

服务	技术/端口	主要职责	持久化与依赖
Frontend	React 18、Webpack 5; 3301	观测页面、AI 对话、A2UI 渲染、Loop 画布、设置与管理界面	无服务端状态；依赖 Query Service 和 Agent Service
Query Service	Go 1.22、Gorilla Mux; 8080/8085	V3/V4 查询、服务和基础设施 API、仪表盘、告警规则、用户组织与 RBAC、集成、日志管道	ClickHouse; 本地 SQLite; Alertmanager
Agent Service	Node.js 22、Hono、Mastra、AI SDK、MCP; 4111	模型适配、对话流、TopoMem、证据链、MCP 工具、安全决策、A2UI、scry-ops 策略与 Loop 编译	PostgreSQL、WASM SQLite、Query Service、外部模型端点
OTel Collector	OpenTelemetry Collector; 4317/4318	接收 OTLP 指标、日志和链路，执行解析、资源补充、批处理与导出	ClickHouse; Query Service 管理接口
ClickHouse	24.1.2	高吞吐遥测写入、时间范围过滤、聚合和高基数查询	clickhouse_data; 依赖 ZooKeeper
Alertmanager	0.23.7; 9093	告警状态和通知通道执行	alertmanager_data; 回调 Query Service 内部 API
Schema Migrator	0.111.16	同步与异步创建/升级 ClickHouse 模式	一次性任务，位于 ClickHouse 就绪之后
ZooKeeper	3.7.1	ClickHouse 协调与集群元数据	zookeeper_data

入数据库内部，可以避免模型直接生成任意 SQL，也使 RBAC、查询格式、缓存、超时和结果大小限制集中在统一边界上。

四条路径通过稳定标识关联。遥测记录使用 service、trace、span、timestamp 和属性集合建立上下文；平台对象使用 user、org、dashboard、rule 等标识；智能执行使用 threadId、runId、stepId、toolCallId 和 workflowId。一次故障诊断可以从告警对象出发，经服务和时间窗口查询到链路和日志，再把证据、模型结论、审批和 SSH 审计关联到同一个 Run。

E. 四平面职责

- **接入平面**：负责 OTLP 接收、数据校验、资源属性补充、批量处理、模式适配和写入重试。
- **查询平面**：负责查询解析、权限校验、查询构建、缓存、数据库执行、后处理和流式进度。
- **智能平面**：负责 Prompt 组装、模型调用、工具选择、Loop 编排、调度、审批和结果持久化。
- **呈现平面**：负责页面路由、任务管理、实时消息、图表、表格、A2UI、配置和运维交互。

每个平面只访问其必要依赖。Collector 不读取 Agent 数据；Agent 不直连 ClickHouse；浏览器不持有数据库凭据；SSH 主机不需要访问平台数据库。该分离降低了组件

被攻破后的横向移动范围，并使容量扩展可以按写入、查询、模型和前端访问分别进行。

F. 请求身份上下文

浏览器请求携带 Scry JWT。Query Service 在路由级执行 Open、View、Edit、Admin 或 Self 权限判断；Agent Service 先验证登录状态和角色，再把原始 JWT 透传给 Query Service 工具调用。智能执行上下文可表示为

$$C_{req} = (userId, orgId, role, token, threadId, runId, mode). \quad (1)$$

工具实现不自行扩大角色权限。Thread、Run 和审批操作依据 userId/orgId 校验所有权；管理模型、SSH 主机、工具策略和 Loop 定义需要 ADMIN；运行已启用的 Loop 和读取本人运行记录由登录用户完成。由此，模型只负责提出工具调用意图，最终执行权由服务端上下文、Schema 和策略共同决定。

IV. 遥测、查询与存储架构

A. 遥测写入路径

Collector 对外暴露 OTLP gRPC 4317 和 OTLP HTTP 4318。接收器解析 Resource、Scope 和信号数据，处理器补充平台所需属性并按批次聚合，导出器把 metrics、logs 和 traces 写入对应 ClickHouse 数据库。Collector 在 ClickHouse、模式迁移器和 Query Service 健康后启动，写入过程可抽象为

$$E \xrightarrow{\text{OTLP}} C \xrightarrow{\text{process}} D_{m,l,t}, \quad (2)$$

其中 E 为埋点应用或采集 Agent， C 为 Collector， $D_{m,l,t}$ 分别表示 ClickHouse 内的 metrics、logs 和 traces 数据集。模式迁移器先创建与版本对齐的本地表、分布式表、物化视图和索引结构，因此 Collector 不承担在线建表职责。批处理参数在吞吐、写入延迟和内存之间取平衡；导出失败进入重试队列，并通过 Collector 自身健康与日志暴露状态。

指标以 metric name、temporality、fingerprint、timestamp 和 value 为核心；日志以纳秒时间戳、severity、body、resource attributes、log attributes、traceId 和 spanId 为核心；链路以 traceId、spanId、parentSpanId、service、operation、duration、status、events 和 attributes 为核心。统一的 Trace/Span 标识使日志、异常和链路详情能够互相跳转。

B. 统一查询路径

Query Service 的 /api/v3/query_range 接收统一查询结构，分别调用 metrics、logs 和 traces 构建器。一次请求经过六个阶段：参数与时间范围校验、信号类型分派、查询语句构建、缓存判定、ClickHouse 执行、后处理与响应格式化。后处理支持表格化、缺口填充、公式、Having、Limit 和 Reduce 等操作，使前端面板与 Agent 工具获得一致结果。

查询层同时提供属性键/值自动补全、聚合属性、过滤候选、查询格式化、查询进度推送和日志 Live Tail。前端 Query Builder、仪表盘、告警规则和 Agent 的 querySignals 工具复用同一查询契约，不维护互相分叉

的查询语义。WebSocket 或流式端点负责长查询进度，用户中止操作会取消前端消费并释放后续处理资源。

设请求时间窗口为 $[t_s, t_e]$ ，序列数为 n ，每条序列的采样点为 p_i ，则结果规模近似为

$$S \propto \sum_{i=1}^n p_i, \quad p_i \approx \frac{t_e - t_s}{\Delta_i}. \quad (3)$$

Agent 系统提示要求限制时间范围和结果规模；服务端进一步将单次工具返回文本限制为 SCRY_AGENT_MAX_TOOL_RESULT_BYTES，默认 200,000 字节。这一组合同时控制模型上下文消耗和意外的大查询传播。面向图表的查询通过步长和最大点数控制采样密度，面向表格的查询通过 Limit 控制行数，面向日志和链路的明细查询通过排序游标实现稳定翻页。

C. 存储职责分离

ClickHouse 只保存遥测分析数据。Query Service 的 SQLite 保存用户、组织、仪表盘、规则和其他业务对象。Agent 的 WASM SQLite 文件保存模型设置、UI 消息、MCP 插件、SSH 主机和策略、安全审计、工作流定义和运行状态；Mastra Memory 使用 PostgreSQL 的独立 Schema。各存储不应混为同一备份单元：恢复可观测数据需要 ClickHouse，恢复平台配置需要 Query Service 数据库，恢复 Agent 对话和密钥密文需要 Agent SQLite、PostgreSQL 与同一个主密钥。

D. 数据生命周期与一致性

遥测数据按照平台 TTL 设置在 ClickHouse 中自动淘汰，配置对象则长期保存并通过显式删除或版本升级管理。Thread 删除会级联移除 UI Message 和关联 Loop Run；Workflow 删除会先清理运行记录；Workflow 定义变更会把旧定义写入版本历史。API Key 和 SSH Secret 只保存密文，读取设置时仅返回 hasApiKey/hasSecret 状态。

实时流和持久化采用双分支处理：一个分支立即返回浏览器，另一个分支消费完整 UI Message Stream 并写入消息表。工作流步骤在开始、暂停、完成和失败时更新运行记录，因此页面刷新后仍能恢复任务历史。调度器为每次触发生成唯一 trigger key，结合 workflowId 和计划时间或事件 ID 做幂等检查，避免多副本或重复事件创建重复 Run。

V. 完整功能说明

A. 观测与分析功能

表 III 根据前端路由和 Query Service API 汇总面向用户的观测功能。

B. 平台管理功能

C. 应用性能监控

服务页以 service name 为主索引汇总应用性能。列展示请求速率、错误率和延迟分位数，并允许按环境、版本、集群及自定义 Resource Attribute 筛选。服务详情把时间范围作为共享上下文，同步驱动吞吐、错误、P50/P90/P99 延迟、端点列表和依赖关系。用户从异常曲线选择时间区间后，可以直接进入对应端点、链路或日志，避免重新输入过滤条件。

TABLE III
观测、分析与告警功能

功能域	入口/对象	完整能力	状态
服务监控	/services、服务详情	服务列表；RPS、错误率、P50/P90/P99 延迟；端点和顶层操作；基于 span 的服务指标	已实现
链路分析	Traces Explorer、Trace Detail	条件过滤、属性补全、保存视图、链路详情、跨度瀑布/火焰关系、Trace ID 定位	已实现
日志管理	Logs Explorer、Live Logs、Pipelines	日志查询与聚合、实时尾随、字段管理、保存视图、解析管道预览与版本化配置	已实现
指标查询	Query Builder、仪表盘组件	指标元数据、PromQL/查询构建、公式和后处理、时间序列/表格可视化	已实现
基础设施	Hosts 及 API	主机、进程、Pod、PVC、Node、Namespace、Cluster、Deployment、DaemonSet、StatefulSet、Job 的列表和属性筛选	已实现
服务拓扑	/service-map	根据链路依赖构建服务关系图，用于识别上下游与故障传播路径	已实现
异常	Exceptions、Error Detail	异常分组、计数、详情、前后错误导航及与链路上下文关联	已实现
消息队列	Kafka 页面与详情	Consumer Lag、生产/消费吞吐、分区延迟、网络延迟和接入状态	已实现
仪表盘	列表、详情、Widget	创建、读取、编辑、删除仪表盘；变量查询；多种面板与自定义查询	已实现
告警	Rules、Overview、History	规则 CRUD/测试、状态时间线、统计、Top Contributors、维护停机计划	已实现
通知通道	Settings/Channels	通道列表、创建、编辑、删除和测试；由 Alertmanager 执行通知	已实现
用量与状态	Usage、Status	遥测用量、版本、健康状态、功能标志和运行配置查看	已实现

TABLE IV
平台、团队与智能运维功能

功能域	能力	状态
接入引导	首次注册、组织初始化、应用/日志/基础设施/AWS/Azure 监控接入步骤	已实现
身份与团队	登录、邀请与批量邀请、用户和组织管理、密码重置、个人设置、ADMIN/EDITOR/VIEWER 权限	已实现
访问与摄取	Access Token、Ingestion Key、数据保留时间 (TTL)、Apdex 阈值、字段配置	已实现
集成	内置集成安装与配置；ClickHouse、云平台和消息队列等数据源接入	已实现
调试模式	管理员启停调试数据、生成和清理 OTLP 验证数据	已实现
AI 助手	多任务对话、诊断/规划/操作模式、附件、流式推理、工具结果、来源、重试与停止	已实现
证据工作台	工具证据卡片、置信度、关联度、关系、接受/驳回/挂起/备注/选择及基于更新证据重答	已实现
TopoMem	运行拓扑检索、错误签名、分块预览、来源关系、检索解释、证据写回、自学习与自维护	已实现
Loop	可视化节点编排、条件/并行/循环、审批暂停恢复、输入、标签、版本和运行记录	已实现
Loop 调度	Cron、时区、事件类型、过滤器、幂等触发、自动运行和失败重试	已实现
AI 设置	模型供应商、官方/兼容端点、API Key、生成参数、系统补充指令、记忆窗口	已实现
SSH 安全	主机指纹、密码/私钥密文、连接测试、只读白名单、预配置命令、逐次审批、审计	已实现

顶层操作用于把框架级 Span 聚合成可理解的入口操作，例如 HTTP Route、RPC Method 或消息消费处理器。系统计算每个操作的调用量、错误率、平均/分位延迟和对服务整体延迟的贡献。Apdex 阈值由管理员设置，Query Service 按满意、可容忍和失败区间计算服务体验分数。服务详情同时保留原始属性和值的跳转能力，便于定位某个版本、区域或客户分组下的性能退化。

D. 分布式链路

Traces Explorer 支持时间范围、服务、操作、持续时间、状态、Span Kind 和任意 Resource/Span Attribute 的组合筛选。查询结果提供 Trace 列表、持续时间分布和聚合统计；保存视图将查询、列配置、排序和展示模式持久化。Trace Detail 读取完整跨度树，展示父子关系、事件、错误状态、属性和时间轴，并根据相对起止时间绘制瀑布或火焰关系。

链路详情支持跨信号关联。Span 中的 service、traceId、spanId 和时间范围可以生成上下文链接，跳转到同一请求

附近的日志；异常记录可以回到产生异常的 Span；服务拓扑使用聚合后的调用边构建上下游关系。对于高延迟请求，用户可以按自身耗时、子跨度耗时和外部依赖耗时区分瓶颈所在层级。

E. 日志检索与处理

Logs Explorer 提供结构化 Query Builder、自由文本条件、字段选择、排序、直方图和表格明细。属性自动补全区分 Resource、Scope 和 Log Attribute，并根据字段类型提供字符串、数字和布尔运算。Live Logs 使用流式接口持续接收最新日志，支持暂停、恢复和保持当前过滤条件。保存视图记录查询语句、时间偏移、列宽和展示字段，供团队复用。

日志解析管道用于把非结构化 Body 转换为可查询字段。管理员可以创建管道、配置匹配条件和解析算子、预览样本结果、保存新版本并控制启用顺序。字段管理允许把高频字段提升为索引字段，系统在更新前校验名称、类

型和冲突。日志记录保留 `traceId/spanId` 时，界面直接提供链路跳转；错误日志还可以进入异常聚合视图。

F. 指标与统一查询

指标查询支持时间序列、标量、表格和公式。Query Builder 选择 `metric`、聚合、过滤、Group By、函数和步长，后端根据 `temporality` 区分 `Gauge`、`Cumulative` 和 `Delta` 语义，并对 `Counter` 计算 `rate/increase`。多查询结果通过公式节点进行四则运算或比例计算，后处理完成缺口填充、单位转换、排序、Having 和 Limit。

统一 V3/V4 查询契约让相同查询对象可以用于仪表盘面板、告警评估、服务分析和 Agent 工具。属性补全接口只返回当前信号和时间范围内存在的键值；查询格式化接口用于展示最终执行语义；查询进度接口提供长查询状态。结果携带单位、时间戳、序列标签和元数据，前端根据面板类型选择 `uPlot`、表格、数值、直方图或其他可视化组件。

G. 基础设施监控

基础设施模块覆盖 Host、Process、Pod、PVC、Node、Namespace、Cluster、Deployment、DaemonSet、StatefulSet 和 Job。每类资源都提供属性键、属性值和列表 API，列表请求包含时间窗口、过滤、排序和分页。主机视图重点展示 CPU、内存、磁盘、网络、负载和进程；Kubernetes 视图聚合资源请求、限制、实际使用、重启、状态和工作负载关系。

资源详情把基础设施指标与服务信号关联。例如 Pod 可以关联 `service.name` 和 `deployment/environment`，Node 可以查看其上运行的 Pod，Deployment 可以比较副本与版本。基础设施告警使用同一规则系统，Agent 也可以先列出服务和告警，再通过统一查询读取特定资源指标，形成从平台信号到主机巡检的闭环。

H. 服务拓扑与异常管理

服务拓扑根据链路中的调用方向聚合节点和边。节点包含服务健康、请求量、错误率和延迟，边包含调用次数、错误率和跨服务延迟。时间范围或过滤条件变化会重新计算拓扑，用户可以从边直接进入调用链路，从节点进入服务详情。该视图适合识别共享依赖、扇入扇出、循环调用和故障传播路径。

异常模块按异常类型、消息、堆栈和服务上下文生成 Group。列表显示首次出现、最近出现、次数、受影响服务和趋势；详情展示代表样本、堆栈、属性、关联 Trace 和相邻错误。通过 Group ID、Error ID 和前后导航接口，用户可以在聚合视图与原始事件之间切换，并把异常条件保存为查询或告警。

I. 消息队列监控

Kafka 监控覆盖 Consumer Lag、Producer/Consumer Throughput、Partition Latency 和 Network Latency。系统按 `cluster`、`topic`、`partition`、`consumer group`、`client` 和 `broker` 组织指标，既提供全局概览，也提供生产者、消费者和分区明细。接入页面验证采集状态，详情页支持按时间范围比较积压增长、消费速率和端到端延迟。

消息队列信号可以和服务、链路、日志及告警组合使用。当某个 `consumer group` 的 Lag 持续增长时，用户可以定位对应消费者服务，查看消费 Span、错误日志和实

例资源；Agent 也可以把积压、服务错误率和主机负载放在同一诊断计划中。

J. 仪表盘与告警

仪表盘支持列表、创建、复制、编辑、删除、变量和独立 Widget 页面。面板保存查询、公式、时间偏移、单位、阈值、图例、布局和显示选项；变量通过查询接口动态生成值，并注入所有引用该变量的面板。布局使用响应式网格，编辑状态和查看状态分离，避免误操作改变生产看板。

告警规则复用统一查询对象，并配置评估窗口、频率、阈值、恢复条件、标签、注释和通知通道。规则管理器定时执行查询，维护 Pending、Firing 和 Resolved 状态，并把状态发送给 Alertmanager。历史页面展示状态时间线、统计、Top Contributors 和总体转换；停机计划在指定窗口抑制通知但保留评估记录。通道管理支持创建、编辑、删除和发送测试消息。

K. 集成、身份与平台设置

集成模块管理内置接入模板和已安装实例，覆盖数据库、基础设施、云服务和消息队列等场景。模板定义所需遥测、接入步骤、默认仪表盘和验证条件。安装完成后，系统根据实际采集信号显示健康状态，并把相关仪表盘和告警入口组织到同一集成详情中。

身份系统支持首个管理员注册、登录、邀请、批量邀请、邀请撤销、用户编辑、角色修改、密码重置和个人偏好。角色分为 ADMIN、EDITOR 和 VIEWER：ADMIN 管理组织、用户、AI、SSH 和 Loop；EDITOR 创建和修改观测对象；VIEWER 执行查询和查看已授权资源。组织设置管理名称、偏好、数据保留、Apdex、Ingestion Key 和 Access Token。

接入引导按应用监控、日志管理、基础设施、AWS 和 Azure 等场景提供步骤化配置。连接状态页面通过查询实际数据确认接入，而不是只检查配置存在。调试模式通过现有 OTLP 链路生成二十类诊断案例，覆盖健康基线、PostgreSQL 连接池与表膨胀、Valkey/Redis 连接上限与持久化延迟、Kafka 消费积压、TLS 证书过期、DNS 解析失败、gRPC 截止时间、HTTP 429、响应结构不兼容、内存泄漏与 OOM、CPU 和线程池饱和、磁盘空间与 I/O、配置漂移、服务崩溃、网络监听暴露、僵尸进程以及复合故障。

场景目录把用户症状、服务拓扑、根因节点、错误比例、异常事件、根服务日志、上游传播日志、干扰日志、业务指标、主机指标和消息指标组织为同一不可变定义。生成器按轻量、标准和高负载档位分别产生 2、6 和 18 条请求链路，并在同一数据集中保留正常依赖对照；父 Span 覆盖完整下游时长，根因异常落在实际失败节点，上游 Span 只记录传播后的 4xx/5xx 或超时状态。结构化日志携带 Trace ID 和对应服务 Span ID，故障指标同时写入十分钟前的健康基线点和当前故障点，因此 Agent 可以从跨信号相关性和趋势而非单一错误文本完成诊断。

布尔属性 `scry.debug` 用于标记模拟遥测。字符串属性 `scry.dataset` 的值为 `diagnostic-evaluation`，`scry.debug.case_id` 使用不透明编号 `case-xxx`。这些属性不携带场景名称、根因服务或根因标签。公开目录 API 返回编号、名称、类别、难度、症状、拓

扑、信号类型和故障比例；管理员真值 API 返回根服务、标准根因、关键证据、处置、验证点和不安全动作。端到端准确性工具据此自动生成数据、调用 Agent、解析结构化诊断，并按根服务、根因、证据、处置、验证和安全六个维度评分。清理流程只删除带调试标识的对象和自动告警规则，不影响真实遥测。

VI. Query Service 与前端实现

A. Go Query Service

Query Service 是浏览器、Agent 和数据存储之间的领域网关。其 APIHandler 组合 Reader、DAO、Query Builder、Querier、Rule Manager、Alertmanager Manager、Integration Controller、日志解析管道和基础设施 Repository。V3 查询构建器按信号类型委派：指标使用 metrics v3，日志和链路在当前启动参数下使用 v4。HTTP 路由通过访问级别包装器区分 Open、View、Edit、Admin 和 Self 权限。

除读查询之外，Query Service 还管理如下控制对象：仪表盘及变量、Explorer 保存视图、告警规则和停机计划、通知通道、字段与摄取设置、用户与邀请、组织和角色、偏好、集成和日志管道。它同时在 8085 提供供 Alertmanager 等内部组件使用的私有 API。该服务承担“统一语义和统一授权”，因此 Agent 工具层只需暴露少量高层操作。

Query Service 启动时完成配置解析、日志初始化、数据库迁移、Reader/DAO 创建、查询缓存初始化、规则管理器启动、集成控制器注册和 HTTP Server 装配。公共端口 8080 面向浏览器和 Agent，私有端口 8085 面向 Alertmanager 等内部服务。健康通道持续报告服务状态，进程接收 SIGTERM 后依次停止 HTTP Server、规则任务和后台资源，保证容器编排可以进行优雅下线。

B. 查询执行管线

统一查询请求包含 signal、query builder expression、时间范围、步长、变量、公式和后处理选项。HTTP Handler 先完成 JSON 解码、角色校验和业务参数校验，再把请求交给 Query Builder。Query Builder 不直接执行 SQL，而是根据 metrics/logs/traces 类型选择专用构建器，生成带参数约束的 ClickHouse 查询计划。Querier 使用缓存键生成器计算稳定 Key，对历史时间段优先读取缓存，对数据仍在流动的尾部窗口绕过缓存。

Reader 执行查询并返回列式结果。后处理层将不同信号的原始结果规范化为统一 Series/Table 结构，依次应用公式、缺口填充、Having、Limit、Reduce 和表格转换。Formatter 根据单位把原始数值转换为时间、字节、数据率、百分比、吞吐或布尔显示。最终响应携带 status、data 和错误类型；超时、取消、坏数据、无权限、冲突和内部错误映射到明确 HTTP 状态。

查询进度通过 WebSocket/流式 API 发布。每个长查询记录 query ID、阶段和完成比例，前端在用户取消或页面离开时停止订阅。Live Tail 使用独立流式路径，持续读取满足条件的新日志。自动补全请求设置短期缓存，减少每次键入都扫描高基数字段。

C. 规则、仪表盘与配置对象

规则管理器把规则定义编译为周期任务。每个任务在评估时间点构造查询窗口，执行统一查询，计算阈值和恢复条件，并将标签、注释和状态传递给 Alertmanager。规则定义和运行任务分离，编辑规则会原子更新任务；停用、删除和维护窗口都有明确状态转换。历史 API 从状态记录中生成时间线、统计和贡献者聚合。

仪表盘、保存视图、通道、用户、组织和偏好通过 DAO 访问 Query Service SQLite。数据库迁移在服务启动阶段执行，确保对象结构与版本一致。写接口使用 Edit 或 Admin 权限，读接口使用 View 权限，用户自身信息和密码使用 Self 权限。所有配置对象通过稳定 ID 访问，删除操作对不存在对象采用幂等语义。

D. 前端应用架构

前端基于 React 18、Redux、React Query、Ant Design、Grafana Data、uPlot/Chart.js 和 Webpack 5。页面按路由懒加载，主导航将 AI 助手和 Loop 放在服务、链路、日志、基础设施、仪表盘、告警等运维入口之前。Agent 页面使用 AI SDK 的 useChat 和 DefaultChatTransport 消费 UI Message Stream；Loop 编辑器使用 DnD Kit 实现节点拖放和排序。

Agent 工作区采用三栏结构：左侧管理任务线程，中间显示对话和输入，右侧检查执行、证据、产物和审批。消息渲染器处理文本、reasoning、来源、文件、工具步骤、A2UI、工作流事件、审批、暂停和 tripwire 等 AI SDK/Mastra 消息部件。表格型 A2UI 最多展示 20 行和 6 列，其他结构化结果回退为 JSON，避免未知结构导致界面崩溃。

应用路由由统一声明页面组件、路径、私有状态和导航 Key。页面组件使用动态 Import 拆分 Bundle，首次进入某个功能域时再加载对应代码。身份信息、组织上下文和访问 JWT 保存在全局状态；服务查询、仪表盘和配置对象使用 React Query 缓存；编辑器局部状态使用组件 Hook 管理。Axios/Fetch 客户端统一注入 API Endpoint、Authorization 和错误处理。

观测页面复用 Query Builder、时间选择器、变量、图表、表格和保存视图组件。用户改变时间范围或过滤条件后，页面生成规范化查询对象，由 React Query 管理加载、缓存、失败和重试状态。图表 Hover、缩放和选区操作会更新共享时间范围，从服务曲线到链路、日志和异常的跳转携带相同上下文。

E. Agent 工作区前端

Agent Workspace 首次加载时读取 Thread、模型设置和已启用 Loop。如果用户没有 Thread，前端自动创建一个。Thread 按“今天”和“更早”分组，可搜索、切换和删除；标题从首条用户消息截取生成。选择 Thread 后，前端读取持久化 UI Message 并恢复工具、审批、来源和工作流事件，而不是只恢复纯文本。

输入区支持多行自动高度、Enter 发送、Shift+Enter 换行、文件选择、文件去重、单文件删除、停止执行和重新生成。选择 Loop 时，请求自动切换到 Workflow Stream Endpoint，并把输入字段序列化为执行消息；普通对话则发送 mode、Thread 和最后一条 UI Message。传输层依据

审批恢复体中的 runId/resumeData 选择继续原 Agent Run 或启动新请求。

右侧 Inspector 把所有 Message Part 分类为执行、证据、产物和审批。工具 Part 展示状态、输入、输出与 A2UI；Source Part 展示 URL 或文档来源；Workflow Part 展示步骤完成比例、输入、输出和暂停载荷；Approval Part 提供拒绝与批准按钮。所有未知 data Part 仍可通过 JSON Details 展示，保证协议新增字段不会丢失。

F. Loop 编辑器前端

Loop Manager 由管理台和编辑器组成。管理台支持搜索、状态/触发方式筛选、创建、复制、删除和进入编辑。编辑器左侧是节点面板，中间是可排序画布，右侧是 Loop 或节点属性。用户可以拖入 Agent、服务、告警、仪表盘、审批、条件、并行和循环节点，并在嵌套编辑器中配置子步骤。

Loop 级配置包括名称、描述、启用状态、标签、Cron、时区、事件触发器和输入 Schema。节点级配置包括名称、提示词、审批文案、重试次数、条件路径、操作符、分支、并行分支、循环模式、最大次数、数据源路径和 foreach 并发。保存前前端保证至少一个节点；服务端继续校验 ID、数量、嵌套深度和类型完整性。运行历史显示状态、时间和输入，用户可以把历史输入重新带入 AI Workspace 执行。

VII. Agent 子系统实现

A. 组件构成

Agent Service 按职责由 Hono HTTP 边界、身份适配器、模型适配器、Mastra Agent 与会话 Memory、TopoMem 运行记忆引擎、证据引擎、MCP Server/Client、Scry/SSH 工具、安全策略、A2UI、Loop 编译器和双存储 Repository 组成。Mastra 提供 Agent 和 Workflow 抽象[4]、[5]，AI SDK 提供模型供应商适配、工具调用和 UI Message Stream[6]，Hono 提供轻量 HTTP 运行时[7]。PostgreSQL 保存 Mastra 会话 Memory 与运行状态，WASM SQLite 保存证据、TopoMem、Loop、插件、SSH 配置和审计，因此 Agent 在 LoongArch64 上不加载本地原生 SQLite 扩展。

HTTP 边界只负责认证、Schema、所有权和响应协议；Agent 负责推理和工具选择；Workflow 负责确定性控制流；Coordinator 负责把时间和事件转换为 Run；Repository 负责定义、状态、恢复点和审计。该划分防止模型逻辑直接操作数据库或调度器，也让手动、定时和事件入口共享同一个 Workflow Runtime。

B. TopoMem 运行拓扑记忆引擎

1) 技术定位与多跳词汇鸿沟：传统会话 Memory 主要保存最近消息，平面 RAG 主要依据查询与文档的词汇或语义相似度排序。微服务事故中，用户看到的症状和根因文档往往由不同团队描述：入口服务使用 HTTP 500、页面延迟、请求超时等词汇，而根因服务的运行手册使用连接池耗尽、锁等待、证书过期或消费位点等领域词汇。两端可能几乎没有共同词汇，真正连接它们的边只存在于本次事故的 Trace。TopoMem 针对这一多跳词汇鸿沟，把事故时运行拓扑作为检索控制信号，而不是把调用链仅拼接成普通查询文本[1]。

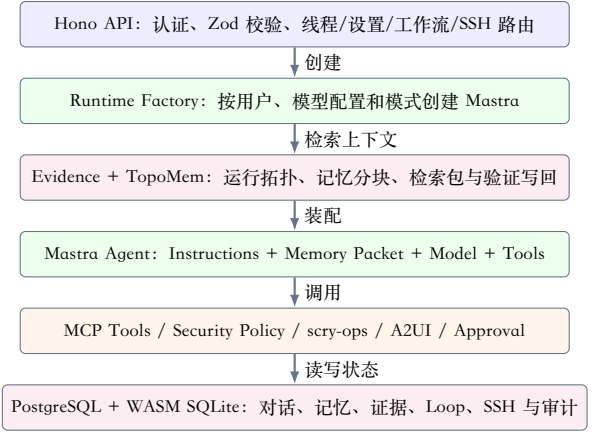


Fig. 3. Agent Service 内部组件。运行时按请求创建，持久化存储在请求之间共享。

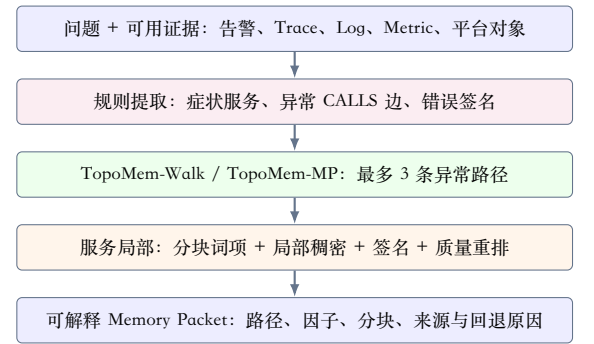


Fig. 4. TopoMem 在线检索管线。拓扑决定优先服务区域，文本信号决定区域内记录顺序。

Scry 中 TopoMem 位于 Agent 创建阶段。Runtime Factory 读取当前 Thread 中状态可用的证据，过滤已驳回和已挂起项目，从证据正文提取 Trace 边、服务名和错误签名，再执行记忆检索。返回的 Memory Packet 在模型首次选择工具之前注入系统上下文，但被明确标记为“不可信历史经验”：它只能用于形成待验证假设和选择首个服务目标，不能替代本轮遥测证据，也不能被当作证据 ID 引用。

2) 运行时拓扑提取与路径导航：TopoMem 从两类结构提取有向边。第一类是具有 spanId、parentSpanId 和 service.name 的 Span 集合，父子 Span 跨服务时形成调用边；第二类是工具结果中显式的 fromService/toService、upstream/downstream 或 caller/callee 结构。边记录调用次数 c_e 和错误率 r_e ，只有 $r_e \geq 0.1$ 的边进入异常子图。症状服务优先取用户问题中出现的服务名，其次取告警或服务证据中的首个服务，最后回退到第一条运行边的起点。

从症状服务开始，路径枚举对每个节点最多展开 3 条出边，单条路径最多包含 6 个服务节点，并保留累计权重最高的 3 条路径。出边和路径使用

$$W(p) = \sum_{e \in p} r_e \ln(1 + c_e) \quad (4)$$

排序，使高错误率且具有足够调用样本的下游边优先于

偶发单次错误。到达异常子图叶节点或深度上限即停止。每次提取到的边还会写入 `scry\_memory\_topology\_edges`, 累计观察次数、错误观察次数、平均错误率、置信度、最近 Trace 和时间; 在线检索仍以本次证据中的运行边为准, 历史拓扑不会伪装成本次事故中真实存在的调用关系。

对于路径 $p = (s_0, \dots, s_n)$ 上的服务 s_i , 拓扑得分定义为

$$T(s_i, p) = 35 + 65 \frac{i}{n}, \quad n > 0, \quad (5)$$

因此越靠近异常路径终端的服务得到越高优先级, 非路径服务得分为 0。该设计把“应该先查哪个服务”和“该服务可能是哪一种故障模式”拆成两个阶段, 避免入口症状文档因词汇重合而长期压过根因服务记录。

3) 错误签名与稳定记忆分块: 错误签名由确定性规则提取, 不调用模型。提取器读取问题文本及可用证据中的标题、摘要和结构化正文, 识别 HTTP 状态码、gRPC/SQLSTATE 标识、以 error、exception、timeout、exhausted、refused、unavailable、overflow、deadlock 和 throttling 等模式结尾的错误词, 以及“错误、异常、失败、超时、耗尽、拒绝、限流、熔断、延迟、重试”等中英文关键词。签名最多保留 24 项, 既用于单路径终端支持校验, 也用于多路径分支选择和候选分块重排。

每条记忆由摘要和结构化正文生成稳定分块。系统优先按段落和句子边界切分, 单块不超过 1,800 字符, 单条记忆最多 80 块; 块类型自动归类为 summary、observation、procedure、evidence 或 metadata。每块保存 SHA-256、词项数、关键词、服务名和置信度, 重复内容按哈希去除。长记忆因此可以在 UI 中逐块预览, 检索也只把得分最高的 3 个块放入 Memory Packet, 避免整篇文档挤占模型上下文。

对查询 q 和记忆块 b , 系统计算词项 Jaccard 相似度 $L(q, b)$ 、256 维哈希特征余弦相似度 $D(q, b)$ 和错误签名覆盖率 $G(b, \sigma)$ 。块内排序为

$$B(q, b) = 0.45L + 0.35D + 0.20G. \quad (6)$$

该局部稠密通道完全在 Agent Service 内执行, 不依赖外部 Embedding 服务; 它作为中英文词形差异的补充, 而不是替代运行拓扑。

4) TopoMem-MP 与选择性回退: 当异常子图只有一条候选路径时, 系统使用 TopoMem; 存在多条路径时使用 TopoMem-MP。TopoMem-MP 分别统计每条路径终端服务记忆对错误签名的最大支持度: 若某一路径获得唯一正支持, 只保留该路径; 若多个分支同分, 保留并融合候选; 若所有终端都没有签名支持, 则放弃拓扑约束并进入词项回退。该机制用于处理重试、并发故障、旧错误分支和低流量噪声, 避免简单贪心始终追随错误率最高的分支。

选择性回退还覆盖三种情况: 当前证据没有可解析 Trace; 存在边但无法从症状服务形成异常路径; 单路径终端没有记忆, 或终端记忆的签名/局部稠密支持低于 5 分。每次回退都返回明确 fallbackReason, 不会静默把平面检索描述成拓扑定位。回退模式继续使用词项、局部稠密、签名和质量因子, 使没有 Trace 的日志类事故仍在使用历史经验。

5) 多因子评分与可解释检索包: 候选质量分综合记忆置信度 C 、重要性 I 、活性 V 和验证状态:

$$Q = 0.35C + 0.25I + 0.30V + 10\mathbb{I}_{\text{verified}}. \quad (7)$$

TopoMem 模式的最终分数和词项回退分数分别为

$$S_{\text{topo}} = 0.58T + 0.17G + 0.09L + 0.07D + 0.09Q, \quad (8)$$

$$S_{\text{fallback}} = 0.46L + 0.34D + 0.10G + 0.10Q. \quad (9)$$

结果先按最终分数、再按活性排序, 默认返回 5 项, 接口允许 1–30 项。每个结果同时返回 lexical、dense、topology、signature、quality 和 final 六个因子, 匹配路径、命中签名、前三个记忆分块及其独立得分也一并返回。检索运行写入 `scry\_memory\_retrieval\_runs`, 记录 strategy、symptomService、signature、paths、fallbackReason、候选数量、返回记忆 ID、耗时、Thread 和 Trace, 使一次“为什么先查这个服务”的选择可以被复盘。

6) 验证写回、自学习与自维护: TopoMem 仓库区分 working、episodic、semantic 和 procedural 四类记忆, 以及 organization、user、agent、thread 和 workflow 五种作用域。组织级写入仅允许管理员; 线程和 Loop 记忆必须绑定对应 ID。系统提供三条学习路径: 用户或 Agent 通过 remember 把选定证据写回; 证据被人工接受后自动生成 verified 经验; Loop 完成后把结果与可用证据共同写为 episodic 或 procedural 记忆。已驳回或挂起证据不能作为来源, 来源表保存 evidence、message、workflow、manual、memory 或 document 标识、Trace、关系和权重。

写入前执行秘密遮蔽、内容稳定序列化、120,000 字节上限和 Prompt Injection 检测。相同内容哈希再次出现时不创建副本, 而是增加强化次数、活性、置信度和重要性; 新记忆会与同组织最近记录建立 duplicates、conflicts、supports 或 same-service 关系。用户可以固定、验证、挂起、归档或遗忘记忆; 遗忘会清空正文、摘要、标签、关键词、服务标识和分块, 仅保留不可混淆的审计骨架。

维护器为不同类型设置不同半衰期: working 为 1 天, episodic 为 90 天, semantic 为 365 天, procedural 为 540 天。未固定且未验证记忆的活性按

$$V = 0.34I + 0.32C + 0.24R + 0.10F \quad (10)$$

重新计算, 其中 $R = 100 \cdot 0.5^{d/h}$ 表示距最近访问或强化 d 天后的时效分, h 为类型半衰期, F 由强化次数和访问次数共同生成。活性低于 20 的在线记忆自动归档; 相似度不低于 0.88 的重复记忆合并并强化保留项; 同主题出现明显相反状态时建立 conflicts 关系, 并把较低置信度且未验证的记忆挂起等待人工复核。维护报告返回扫描、衰减、强化、合并、冲突、归档和过期数量, 并写入安全审计。

记忆管理页面展示总量、在线量、验证量、挂起量、平均活性、类型和状态分布; 详情视图展示正文、分块预览、关键词、来源、关系和完整维护历史。执行 TopoMem 检索后, 页面进一步显示检索策略、路径、签名、六因子得分和命中块, 因此记忆不是隐藏在 Prompt

中的黑盒上下文，而是可以逐项检查、删除、修订和重新验证的运行资产。

C. MCP 工具协议与插件注册

Agent 的实际工具调用链通过 Model Context Protocol 完成。内置 /mcp 端点采用 Streamable HTTP 传输，每个请求使用当前 Scry Bearer Token 重新解析用户、组织和角色，再创建请求作用域的 MCP Server。Server 当前发布 16 个工具，覆盖计划、服务、告警、仪表盘、Query Range、安全意图、SSH、证据链与 TopoMem。MCP Client 在创建 Agent 时执行 initialize 与 tools/list，将返回的 JSON Schema 转换为 Mastra 工具，并在对话流或 Loop 节点结束后主动断开连接。

运行时 tools/list 使用 camelCase 名称，例如 querySignals、listEvidence 与 sshReadonlyInspect；装载到 Agent 后由 MCP Client 增加服务命名空间，分别成为 scry_querySignals、scry_listEvidence 与 scry_sshReadonlyInspect。系统指令与技术文档均使用这一运行时映射，避免模型依据内部源码 ID 猜测工具名。

每个 inputSchema 都是 additionalProperties=false 的对象，顶层和嵌套字段包含中文说明、枚举、默认值、边界、引用来源与调用示例。模型必须直接提交一个匹配 Schema 的 JSON 对象，不得再添加参数包装层；无参数工具固定提交空对象。ID 型字段必须来自前置工具真实输出，例如 SSH 主机和命令来自 sshListHosts，证据 UUID 来自 listEvidence，记忆 UUID 来自 searchMemory。校验失败后只修正被指出的字段，不允许切换为底层协议继续试探。考虑到系统支持自定义 OpenAI Responses 兼容端点，Scry 不发送 provider strict 扩展元数据，参数的确定性校验在 MCP Server 工具边界完成。

querySignals 使用单一扁平对象，唯一必填顶层字段 signal 取 logs、traces 或 metrics。公共字段包括回看分钟数、案例编号、服务列表、附加过滤、结果模式、分组和时间桶；日志增加正文搜索与级别，链路增加操作名、仅错误和最小耗时，指标增加指标名、指标类型、聚合和 temporality。模型不得添加外层 query，也不得提交 compositeQuery、logQueries 或 builderQueries。queryType、start 与 end 由编译器生成。

工具执行层把高层输入确定性编译为 Query Range V3：自动计算毫秒时间窗和 step，固定声明 queryType=builder，创建 builderQueries.A，并按记录或时间序列选择 panelType、聚合、排序、分页和字段作用域。日志与链路中的 service.name 映射到资源属性；Trace ID、Span ID、日志级别等别名映射到对应标准列；scry.debug.case_id 也映射到资源属性；其他 OpenTelemetry 字段映射到普通属性。Zod 在网络调用前拒绝缺少 metricName、错误聚合、非数组 in/nin 和越界时间范围，避免模型以底层 400 响应进行协议试探。

内置 MCP Server 在 initialize 响应中发布统一调用协议，Agent 同时使用同一份受版本控制的协议常量构建系统提示。MCP Client 保持 Server Instructions 自动转发关闭，避免自定义 Responses 兼容端点因重复 Guidance 进入空推理循环，并防止第三方内容取得系统指令权限。管理员可以按组织注册远程 MCP 服务。注册记录包含名称、

说明、Streamable HTTP URL、启用状态、超时和可选请求头；请求头使用 AES-256-GCM 加密，列表接口只返回 hasHeaders。Agent 不向外部插件转发 Scry JWT，插件只能获得管理员为该插件单独配置的认证头。远程服务连接失败时仅跳过该插件并记录错误，内置 Scry MCP Server 失败则终止 Agent 创建，避免在缺少核心安全工具时降级运行。

Loop 的 mcp-tool 节点直接引用命名空间化的 toolId 和结构化参数。security-check、operation 与 verify 节点分别承担强制策略解释、审批后受控执行和结果复核。系统初始化提供安全决策、磁盘爆满、僵尸进程、磁盘 I/O、配置漂移、服务恢复、网络暴露面、综合根因和 Prompt Injection 审查预设，其中系统安全 Loop 锁定，不能被修改或删除。

D. 运行时创建与系统指令

createScryAgent(user, settings, mode) 每次请求创建用户相关 Agent，ID 为 scry-operator-`<userId>`。Memory 的 lastMessages 来自管理员配置，资源 ID 为用户 ID，线程 ID 为对话 ID，从而形成

$$K_{memory} = (userId, threadId). \quad (11)$$

该键只表示短期会话 Memory。TopoMem 是独立的长期运维记忆层，按 orgId、userId、scope、serviceName、status 和运行证据上下文执行检索，并把结果压缩为有界 Memory Packet。二者同时存在：短期 Memory 保持对话连续性，TopoMem 负责跨 Thread、跨 Loop 和跨事故复用已验证经验，证据引擎负责阻止两者被误当作当前事实。

系统指令固定要求使用简体中文、复杂诊断先调用 scry_publishPlan 发布 2 至 6 步计划、仅依据工具数据下结论、查询必须有界、严格遵循 MCP inputSchema、不得猜测底层 HTTP API、SSH 只能使用已配置的主机和命令 ID、正文不重复全部 A2UI 原始数据，并采用“结论、证据、下一步”的输出顺序。工具校验失败、工具不可用、无查询结果和确认存在服务故障被定义为不同状态，模型不得把接口错误转换为故障事实。管理员补充指令附加在内置规则之后，不替换基础安全规则。

三种交互模式改变任务姿态：diagnose 优先用最少观测查询定位根因；plan 澄清目标、依赖、风险和执行步骤，不触发外部变更；operate 可以调用管理员启用的操作工具，高风险调用进入人工审批。模式只是行为约束，真正的执行权限仍由工具注册、用户角色、主机状态、命令策略和审批结果共同决定。

E. Prompt 分层

最终 Prompt 由六层内容组成。第一层是不可被管理员覆盖的 Scry 基础规则，包括语言、证据、权限和输出结构；第二层是模式规则；第三层是管理员补充指令；第四层是 TopoMem 检索包及其“必须重新验证”边界；第五层是 Thread 短期记忆和当前用户消息；第六层是本轮工具返回的观测证据。附件、记忆和工具结果都被标记为不可信数据，不被当作系统指令，其中只有第六层可以直接支撑本轮事实结论。

对于 Loop 中的 Agent 节点，编译器把节点 Prompt、用户输入和前序结果组合为节点级 Prompt。前序结果先

序列化并按 50,000 字符截断，避免一个大型分支完全占据上下文。模型输出仅写入当前节点结果，后续节点通过 results.<stepId> 显式引用，减少隐式状态传播。

F. 模型供应商适配

模型设置支持 openai 与 anthropic 两种协议，以及 official 和 custom 两类端点。OpenAI 路径固定使用 Responses API，Anthropic 路径使用 Messages API[8], [9]。自定义 URL 会去除末尾的 /responses 或 /messages 后再由适配器拼接，避免重复路径。

每次生成应用温度、最大输出 Token、最大步骤、失败重试和工具选择策略。toolChoice=auto 由模型判断是否调用工具，none 禁止工具，required 要求至少一次工具调用。maxSteps 同时限制模型与工具的往返次数，maxRetries 只处理供应商级可重试错误，不重复已产生副作用的工具。

模型连接测试使用独立 AbortController 和超时，要求模型只回复“连接成功”，并将失败归类为配置、认证、模型、限流、超时、网络或供应商问题。错误返回前隐藏疑似 API Key 和外部 URL，并截断为 500 字符。连接测试使用当前表单值和已保存密钥的合并结果，因此管理员可以在保存前验证新模型、端点和凭据。

模型配置是运行时读取的实例级设置。新的请求使用最新配置，已经开始的 Run 保持创建时的模型实例和参数，避免中途切换供应商改变同一执行的语义。Endpoint Preview 在前端展示最终请求 URL，便于排查根路径、/responses 或 /messages 重复拼接问题。

G. 对话、附件与流式持久化

对话 API 的执行序列如图 5。线程所有权同时校验 threadId、userId 和 orgId。用户消息和可解析文本附件先进入安全决策链，拒绝结果返回 traceId 且不进入模型；允许或需审批的请求再写入 UI 消息表，创建 MCP 工具运行时并调用 handleChatStream。流被 tee() 分为客户端分支和持久化分支，最终 Assistant UIMessage 以 Upsert 方式写入 WASM SQLite。

每条消息最多 5 个附件，单文件不超过 10 MB。txt/md/json/log/csv 等文本附件不超过 512 KB，并在送入模型前转换为带“用户提供的数据”边界标记的文本。文件名会移除换行和方括号，避免破坏边界结构；其他文件保留为模型文件部件，由供应商能力决定处理方式。Loop 输入只接受结构化文本和 JSON，文件型任务先在普通 Thread 中解析，再把结果传入 Loop。

H. UI Message Stream 协议

Scry 不把模型响应简化为单一文本，而是完整传输 AI SDK UI Message Part。常见 Part 包括 text、reasoning、source-url、source-document、file、step-start、tool-*, data-tool-call-approval、data-scry-workflow-step、data-scry-workflow-result 和 data-tripwire。每个 Part 都有稳定 type，工具 Part 还包含 state、input、output 和 errorText。

前端依据 Part state 显示输入流式、等待审批、执行中、输出可用、失败、暂停和取消。最终 UI Message 以完整 JSON 保存，因此页面刷新后 reasoning、来源、工具

输入输出和审批卡片仍然存在。重新生成请求携带 trigger 和 messageId，后端不会重复保存同一用户消息；审批恢复请求携带 runId 与 resumeData，继续原 Run 而不是创建新 Run。

I. Scry 观测工具与 A2UI

工具使用 Mastra createTool 与 Zod 同时声明输入和输出。输入 Schema 约束模型能够提交的字段，执行器依据认证上下文和工具策略完成二次校验，输出 Schema 则保证模型与前端接收稳定数据结构。表 V 列出 Agent 运行时注册的标准工具。

callScry 始终把当前用户 JWT 放入 Authorization Header，并在网络层设置连接、响应和总体超时。Query Service 返回非 2xx 状态时，工具将响应转换为统一错误对象；响应体超过配置上限时，执行器保留元数据、字段结构、前后样本和截断标记。工具输出同时包含原始 data、展示结构 a2ui、关联标识和执行统计，模型可以依据原始数据推理，用户则获得适合扫描、比较和审批的视觉结果。

J. 工具调用状态机与 A2UI Schema

每个工具调用使用 toolCallId 标识，并依次经历 input-streaming、input-available、approval-requested、output-available 或 output-error 状态。只读且无需审批的工具从输入可用直接进入执行；受控工具先生成审批部件，批准后以相同 toolCallId 恢复。取消、超时和策略拒绝都产生终态事件，前端不会把它们显示为仍在运行。

A2UI 0.1 顶层包含 version、surfaceId、operation 和 root。其中 surfaceId 在 Thread 内唯一，用于替换或增量更新同一结果。operation 支持 replace、append 和 update；根节点类型及其语义见表 VI。所有类型共用标题、数据、敏感字段标记、截断标记、来源和时间范围元数据。

前端渲染前再次验证 A2UI 版本和根节点类型，所有文本按纯内容处理，Markdown 禁止原始 HTML，表格单页按显示上限裁剪。无法识别的类型进入通用 JSON Inspector，并保留原始 Part，保证协议升级期间的数据兼容性。A2UI 与模型正文相互独立：A2UI 用于证据和操作状态，正文用于解释和结论，避免模型复制大段原始数据造成上下文与页面冗余。

K. SSH 执行链路

SSH 纳管由 scry-ops 环境安装、主机登记、指纹核验、私钥加密、策略绑定、连接测试和启用七个步骤组成。受管主机固定使用 scry-ops 公钥账户，不接受 root、UID 0、其他用户名或密码认证。管理员先调用 probe 端点读取服务器公开主机密钥指纹，再通过可信运维通道核验，保存后连接器使用 hostVerifier 对 SHA-256 指纹做常量时间比较。指纹变化会阻断连接并产生安全审计事件，必须由管理员重新核验和更新。

私钥和 passphrase 以 AES-256-GCM 密文保存在 WASM SQLite。运行时只在建立连接前解密到进程内存，连接关闭后释放引用；列表、详情、审计和错误响应都不返回秘密。连接测试也通过固定的 scry-connection-test 包装器和 sshd ForceCommand，不存在测试旁路。命令执行禁

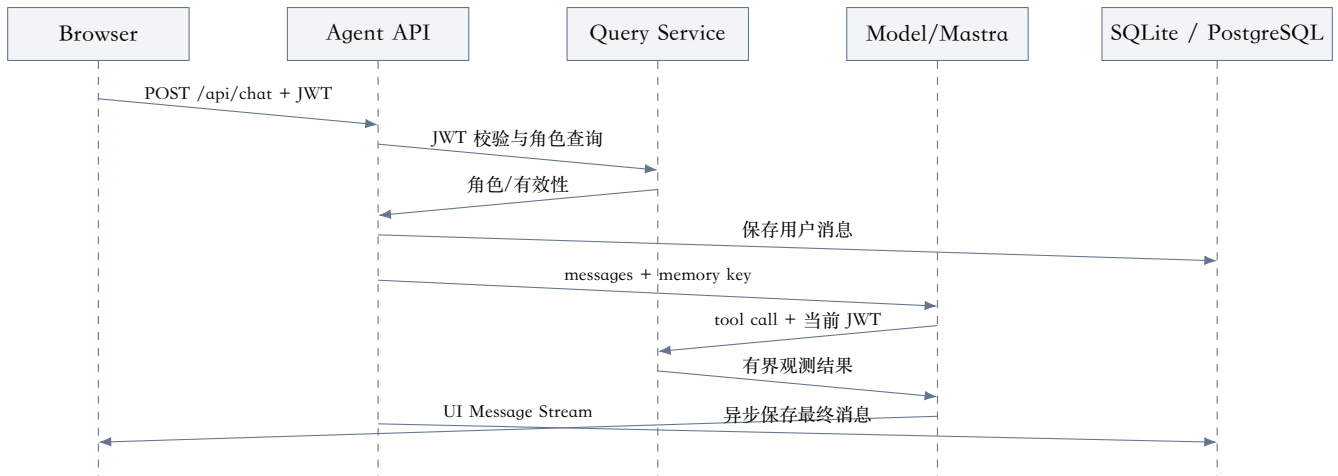


Fig. 5. Agent 对话请求、授权继承、工具取证和流式持久化时序。

用 PTY、交互输入、X11、端口转发和隧道，默认 30 秒超时，标准输出与标准错误合计捕获 200 KB，同时记录真实字节数和截断状态。

SSH 策略分为只读巡检和扩展执行。只读巡检提供七个内置命令 ID，全部映射到 /opt/scry-ops/bin 下由 root 持有的固定包装器：

- system_overview 与 disk_usage;
- memory_usage 与 failed_services;
- network_listeners 与 recent_errors。
- security_preflight 校验账户、UID、附加组、包装器属主和权限。

远端 sshd 对 scry-ops 使用 ForceCommand /opt/scry-ops/bin/scry-dispatch，并禁止 TTY、转发、隧道和用户环境。分发器只接受固定只读包装器或 sudo -n 调用的变更包装器；sudoers 仅允许精确包装器路径，服务名和清理时间等参数在分发器和包装器内各校验一次。扩展命令保存与执行前都经过词法分析，Shell 解释器、管道、重定向、命令替换、提权程序、破坏性程序和非包装器路径一律拒绝。

扩展命令固定进入人工审批。审批卡片包含主机名称、命令名称、风险等级、执行原因、超时和请求用户，不展示凭据。批准结果绑定 runId、toolCallId、hostId、commandId 和策略版本，任何字段变化都会使原批准失效。执行结束后，系统写入用户、组织、线程、运行、主机、工具、命令、策略版本、状态、退出码、耗时、输出字节数、截断标记和错误摘要；审计事件与 Agent Run 的 Trace 标识关联，可从对话、运行历史和安全审计三处定位同一次操作。

L. Loop 定义语言

Loop 是持久化、版本化且可恢复的工作流定义。定义包含基本信息、输入 Schema、触发器和步骤树；步骤树使用版本 1/2 DSL 表示。叶节点负责执行具体能力，控制节点负责组织分支与迭代。

- **叶节点**：推理节点为 agent，MCP 调用节点为 mcp-tool，强制安全节点为 security-check，受控执行和结果复核节点为 operation/verify，

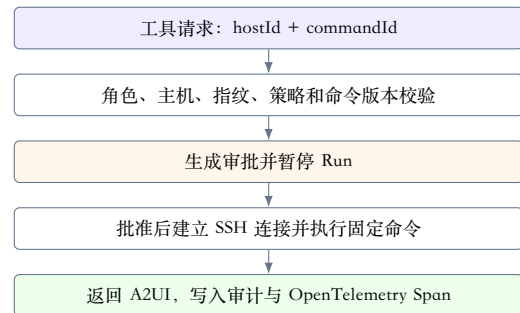


Fig. 6. SSH 受控执行闭环。任一校验失败都会终止调用并写入拒绝事件。

人工控制节点为 approval，观测读取节点为 services、alerts、dashboards。

- **条件节点**：condition 使用路径、操作符和值，分别编译 true/false 子流程。
- **并行节点**：parallel 将 2 至 6 个分支编译为 Mastra parallel，完成后合并上下文。
- **循环节点**：loop 支持 while、until 和 foreach；foreach 可指定源路径和 1-10 的并发数。

条件路径从工作流上下文读取，操作符支持 exists、equals、not_equals、gt/gte/lt/lte。路径以 input、trigger、results、item 或 runtime 为根，比较值按布尔、数字或字符串语义处理。工作流整体最多 100 个节点，控制结构最多嵌套 5 层，所有节点 ID 必须唯一；循环或 foreach 最大 50 次或 50 项，并行分支为 2 至 6 个，foreach 并发为 1 至 10。

Loop 输入字段支持 text、number、boolean 和 json。启动前，运行入口将用户输入、默认值与触发器载荷合并，再按 required 和字段类型校验。运行上下文采用如下结构：

$$C_{loop} = \{input, trigger, results, item, runtime\}. \quad (12)$$

其中 input 在 Run 生命周期内只读，trigger 保存

TABLE V
Agent 工具目录与控制措施

工具 ID	后端操作	用途与输出	控制措施
publishPlan	无外部查询	发布诊断步骤, 生成 plan 型 A2UI	1-8 步; 字段完整描述; 拒绝额外字段
listServices	GET / api/ v1/ services/list	服务清单和 table 型 A2UI	继承 JWT/RBAC; 只读; 空对象输入
listAlerts	GET / api/ v1/ alerts	当前告警和 table 型 A2UI	继承 JWT/RBAC; 只读; 空对象输入
listDashboards	GET / api/ v1/ dashboards	仪表盘清单和 table 型 A2UI	继承 JWT/RBAC; 只读; 空对象输入
querySignals	POST / api/ v3/ query_range	扁平高层格式查询指标、日志或链路, 生成 key-value A2UI	顶层 signal; Schema 预校验; 自动编译 V3; 200 KB 上限
evaluateSecurity Intent	本地策略引擎	返回意图、风险分和 allow/approval/deny	text 与 mode 明确; 不依赖外部模型
listEvidence	读取证据仓库	返回置信度、关联度、状态、选择和备注	默认排除 rejected/suspended
getEvidenceChain	读取证据关系	返回证据节点与印证、派生关系	对话与组织双重隔离
searchMemory	TopoMem 检索	返回根服务优先的候选记忆与评分	query 可省略; limit 为 1-20
remember	写入记忆仓库	写入证据支撑的可复用运行经验	正文为字符串; 证据 UUID 必填; 秘密检测
inspectMemory	读取记忆仓库	返回正文、分块、来源、关系和维护历史	memoryId 来自检索结果
maintainMemory	记忆维护器	执行衰减、强化、去重、冲突和归档	批量上限 500; 不硬删除
forgetMemory	记忆遗忘器	清空可检索内容并保留审计骨架	精确确认字符串 FORGET
sshListHosts	读取 Agent Repository	返回启用主机、只读命令 ID、扩展命令元数据	不返回密钥或精确命令文本
sshReadonly Inspect	执行内置固定命令	系统、磁盘、内存、失败服务、监听端口、近期错误	hostId/ commandId 来自目录; 审批策略; 审计
sshExecute Command	执行管理员预存命令	执行精确命令定义, 返回 code 型 A2UI	拒绝主机名和原始 Shell; 逐次审批; 审计

manual/cron/event 来源与元数据, results 按 stepId 保存步骤输出, item 只在 foreach 子流程中存在, runtime 保存 runId、workflowId、workflowVersion、principal、attempt 和计划时间。显式命名空间使条件表达式和后续 Prompt 不依赖隐含变量。

M. Loop 编译与执行

编译器递归执行 compileSequence(record, definitions, principal, settings, id)。保存 Loop 时先执行结构验证, 启动 Run 时再按固定版本编译, 保证运行中的版本不受后续编辑影响。普通节点通过 then 串联; 条件节点通过 branch; 并行节点通过 parallel; while 和 until 分别映射到 dowhile 与 dountil; foreach 先读取源数组、应用最大项数, 再映射到有界并发子流程。

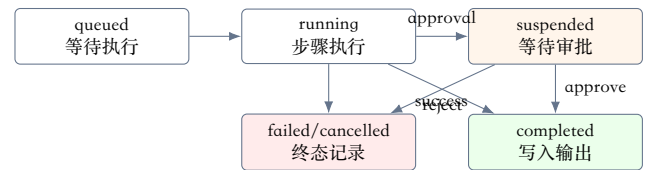


Fig. 7. Loop 运行状态与人工审批恢复。

每个控制节点前后插入标记步骤, 用于发布进度、记录分支选择并把输出归一化为统一上下文。

叶节点执行前发布 data- scry- workflow- step/ running, 并把 attempt、startedAt 和输入摘要写入步骤状态。agent 节点将节点 Prompt、结构化输入和前序

TABLE VI
A2UI 组件语义

类型	渲染与使用方式
plan	展示步骤序号、名称、执行状态和当前步骤，用于诊断计划与 Loop 进度。
table	展示列定义、行数据、单位、排序和分页摘要；大结果提供截断说明。
key_value	展示指标、属性、查询摘要和主机巡检结果，支持单位格式化。
status	展示成功、警告、失败、暂停、取消及其说明。
code	展示预配置命令 ID、退出码和受控 stdout/stderr 片段。
markdown	展示结构化说明、结论和引用，不执行嵌入式脚本。

results 的有界摘要组合为请求；观测节点调用同一个 Scry Read Operation；审批节点调用 Mastra suspend，保存 resume label、审批快照和到期时间。步骤成功后写入 output、duration 和 completedAt；失败时依据节点重试次数分类为可重试或终态错误，并使用指数退避重新入队。

并行分支在独立子上下文中执行，完成后按 branchId 合并；任一分支失败时，控制节点按照 all-success 语义结束并记录各分支状态。while/until 每次迭代后重新计算条件，foreach 为每个 item 生成稳定 itemKey，结果按原数组顺序归并。最终步骤发布 data-scry-workflow-result，写入输出摘要、完整结构化输出和运行统计；与 Thread 关联的 Run 还会把最终文本追加到对话历史。

N. Cron 调度器

Schedule Coordinator 在 Agent Service 启动后加载所有已启用且包含 Cron 的 Loop。Cron 表达式在保存时完成语法校验，时区使用 IANA 标识；调度器将本地日历时间转换为 UTC 计划时间，并在夏令时切换时按时区规则确定唯一触发点。每次 Loop 变更都会刷新内存注册表，停用或删除操作同步撤销任务。

多副本部署使用租约选主。协调器周期性竞争 scheduler-leader 租约，租约包含 holderId、epoch 和 expiresAt；只有持有当前 epoch 的副本可以扫描到期任务。扫描器为每个计划时间生成

$$K_{cron} = H(w, v, t_s), \quad (13)$$

其中 w 为 workflowId， v 为 workflowVersion， t_s 为 scheduledAt。

Trigger Claim 表对 triggerKey 设置唯一约束。领取成功的任务创建 queued Run，领取冲突表示其他副本已经处理，因此直接跳过。服务停机跨过触发点时，恢复扫描按照 misfire policy 处理：skip 跳过期点，fire-once 补执行最近一次，catch-up 在最大补偿次数内逐次创建 Run。Scry 默认采用 fire-once，避免长时间停机后形成执行洪峰。

调度身份在启用 Loop 时固化为 executionPrincipal，包含组织、所有者和权限副本。触发时 Coordinator 向 Query Service 的内部身份端点换取短期 JWT，工具继续通过 Query Service RBAC 访问数据。所有 Cron Run

都记录 scheduledAt、claimedAt、startedAt、principal、schedulerInstance 和 triggerKey，便于定位延迟与重复抑制结果。

O. 事件触发器

Event Coordinator 接收平台内部事件并转换为规范事件信封。事件信封包含 eventId、eventType、occurredAt、orgId、source、subject、attributes、payload 和 traceContext。Query Service、告警管理器及其他集成通过带服务身份的 /api/internal/events 提交事件；接收端验证签名、时间戳和重放窗口后，先写入 Event Inbox，再异步匹配 Loop，保证调用方无需等待模型执行。

每个事件触发器由 eventType、enabled 和 filter 组成。eventType 支持精确名称和受控前缀通配；filter 被编译为只读表达式，只能访问信封字段，支持 equals、not-equals、contains、in、gt/gte/lt/lte 和 exists。保存触发器时完成语法与字段校验，运行时不执行动态代码。匹配成功后生成

$$K_{event} = H(workflowId, triggerId, eventId). \quad (14)$$

唯一 Trigger Claim 保证同一事件对同一触发器只创建一次 Run。事件载荷写入 trigger.payload，同时应用大小上限、敏感字段遮蔽和类型校验。Inbox 消费成功后记录 matched、ignored 或 failed；暂时性失败按退避策略重试，达到次数上限后进入 dead-letter 状态并保留错误分类。管理员可在运行历史中按 eventId 重新投递，重新投递生成新的 replayId，同时保留原始 eventId 关联。

P. 并发控制、幂等与重试

Run Dispatcher 在创建 Run 前应用三层并发控制：实例级总并发、组织级并发和 Loop 级并发。超过配额的 Run 保持 queued，并按优先级、计划时间和创建时间调度。Loop 支持 allow、forbid 和 replace 三种并发策略：allow 允许多个 Run 并行；forbid 在已有活动 Run 时保留一个待执行实例；replace 取消旧 Run 并启动最新实例。

幂等覆盖触发和步骤两个层级。触发层以 Trigger Claim 防止重复创建 Run；步骤层为具有外部副作用的调用生成

$$K_{step} = H(r, s, a, c), \quad (15)$$

其中 r 、 s 、 a 、 c 分别表示 runId、stepId、attempt 和 toolCallId。Repository 先登记 executing，再执行工具，完成后保存输出摘要。供应商网络重试只覆盖尚未产生工具输出的模型请求；SSH 和其他副作用工具不做透明重放，必须依据已保存执行状态人工确认。

错误分为 validation、authentication、authorization、rate-limit、timeout、dependency、provider、tool、approval-rejected、cancelled 和 internal。validation/authorization/approval-rejected 直接结束；rate-limit、timeout、dependency 和 provider 按配置重试；internal 记录 traceId 并由运行器判定。退避时间采用带抖动的指数函数

$$d_n = \min(d_{max}, d_0 2^n) + U(0, j), \quad (16)$$

其中 n 为重试序号, j 为抖动窗口。所有重试沿用同一个 runId, attempt 单调增加, 便于用户区分一次业务运行和多次执行尝试。

Q. 审批暂停与恢复

审批节点进入 suspended 状态时, 运行器提交当前步骤状态、上下文摘要、Workflow Version、resume label 和审批载荷, 随后释放计算资源。审批记录包含 approver scope、requestedAt、expiresAt、decision、comment 和 decidedAt。审批 API 校验运行所有权、审批角色、当前 suspendedStep、版本和状态, 使用条件更新保证同一审批只能成功决策一次。

批准后, /api/workflow-runs/:runId/resume 将 resumeData 写入恢复事件, Run Dispatcher 重新装载固定版本定义和持久化上下文, 从相应 resume label 继续; 拒绝后运行进入 failed, 错误分类为 approval-rejected; 超时后进入 expired。恢复不会重新执行审批点之前已经完成的步骤。服务重启时 Recovery Worker 扫描 running、retrying 和 suspended Run: running 根据最后一个持久化检查点重新排队, retrying 根据 nextAttemptAt 恢复, suspended 保持等待决策。

R. Agent HTTP API

所有 JSON API 返回 requestId。成功响应使用 data 与可选 meta; 失败响应使用 error.code、error.message、error.details 和 traceId。流式接口在 HTTP 建立后通过结构化 Part 报告业务错误, 客户端取消通过 Abort Signal 传播到模型和只读查询。创建操作返回 201, 幂等删除返回 204, 参数错误返回 400, 身份问题返回 401, 权限问题返回 403, 对象不存在返回 404, 版本或状态冲突返回 409, 限流返回 429, 依赖超时返回 504。

S. Agent 持久化模型

Agent Service 的持久化模型按八个领域组织:

- 模型配置: scry_model_settings;
- 对话: scry_agent_threads、scry_agent_ui_messages;
- 证据: scry_evidence_items、scry_evidence_relations、scry_evidence_actions、scry_evidence_revisions;
- TopoMem: scry_memory_items、scry_memory_chunks、scry_memory_sources、scry_memory_relations、scry_memory_actions、scry_memory_topology_edges、scry_memory_retrieval_runs;
- SSH: scry_ssh_hosts、scry_tool_policies、scry_ssh_audit_logs;
- Loop: scry_workflows、scry_workflow_runs、scry_workflow_run_events;
- 触发: scry_trigger_claims、scry_event_inbox;
- 协调: scry_runtime_leases、scry_agent_audit_events。

模型设置以组织和环境为作用域, 密钥只保存密文与 keyVersion。Thread、Message、Run、Workflow、SSH Host 和策略都包含 orgId; 用户访问采用 orgId 与 userId

双重过滤, 平台管理员使用显式管理端点跨用户查询。Workflow 保存当前定义和版本索引, Run 固定引用 workflowVersion; Trigger Claim 以 triggerKey 唯一约束实现幂等; Runtime Lease 使用 name 与 epoch 保证选主正确性。

UI Message 作为完整 JSON 保存, 以适配 AI SDK Part 的扩展。工作流定义、输入、输出、步骤状态、触发器和输入模式也使用 JSON 列, 但关键检索字段保持独立列, 包括 orgId、userId、workflowId、runId、status、triggerType、scheduledAt、nextAttemptAt、createdAt 和 updatedAt。写入路径同时经过 Zod 校验、数据库约束和状态转换检查; Run Event 采用只追加模型, 保存每次状态变化的序号、事件类型、时间、主体、摘要和 traceId。

Repository 使用短事务更新 Run 状态和追加事件。状态更新带 expectedStatus 条件, 防止两个工作进程同时推进同一 Run; Trigger Claim 和租约更新依赖唯一键与 compare-and-swap 语义。大工具结果不重复写入事件表, 事件只保存摘要和内容哈希, 完整结果保存在 UI Message 或 Run Output 中。

VIII. 安全架构

A. 身份认证与请求验证

浏览器通过 Scry 登录流程获得 Bearer JWT, JWT 声明遵循 RFC 7519[10]。Query Service 负责签名、有效期、颁发方、受众和用户状态验证; Agent Service 在本地完成令牌格式检查后, 使用同一令牌向 Query Service 执行在线身份确认。除 /health 外, 所有 Agent API 都要求有效身份; Query Service 不可达、令牌过期、用户停用或组织不匹配时一律拒绝请求。

内部调用使用独立服务身份。Agent 向 Query Service 发起用户数据查询时透传当前用户 JWT; Cron 和事件 Run 通过 executionPrincipal 换取短期组织级 JWT; Query Service、Alertmanager 和集成组件调用事件入口时使用服务令牌和请求签名。用户令牌与服务令牌具有不同 audience, 不能互换。每次访问都重新验证主体、资源和动作, 符合零信任架构的持续验证原则[11]。

B. 授权与租户隔离

平台角色为 ADMIN、EDITOR 和 VIEWER。VIEWER 可以查询遥测、查看仪表盘并运行获得授权的 Loop; EDITOR 可以创建和修改观测对象; ADMIN 可以管理用户、组织、模型、SSH、工具策略、Loop、调度器和审计。高风险操作还需满足工具级策略和人工审批, 角色不会绕过审批。

所有业务对象带 orgId。Thread 和 Run 同时带 userId, 用于个人任务隔离; Workflow、模型设置、SSH 主机和策略按组织管理; 系统级管理员通过独立管理端点执行跨组织运维。Repository 查询把 orgId 作为强制条件, API 不接受客户端覆盖认证上下文中的组织。对象 ID 使用不可预测 UUID, 所有权校验仍然在读取和写入时执行, 避免把随机 ID 当作授权机制。

C. Prompt 与工具安全边界

Scry 将系统规则、模式规则、管理员指令、用户输入、附件、观测数据和工具输出放入不同语义边界。附件和遥

TABLE VII
Agent Service API 分组

分组	端点	授权与语义
健康模型	GET /health GET/ PUT / api/ settings/ model; POST /test	唯一免认证端点 读取隐藏密钥；更新和测试仅 ADMIN
SSH 主机	GET/POST /api/ssh/hosts; probe、test、PUT、DELETE	全部仅 ADMIN；指纹探测、连接测试均有显式 API
SSH 策略/审计	GET/PUT /api/ssh/policies; GET /api/ssh/audits	仅 ADMIN；审计查询上限 200 条
线程	GET/ POST / api/ threads; GET/ DELETE /:threadId	按用户和组织隔离，删除级联 UI 消息与关联运行
对话证据	POST /api/chat GET / api/ threads/:threadId/ evidence; PATCH / api/ evidence/:evidenceId; POST / evidence/revise	线程所有权校验；诊断/规划/操作；流式输出和恢复 自动回填、状态/选择/备注更新和基于选定证据重新回答
TopoMem	GET/ POST/ PATCH/ DELETE / api/ memories; search、stats、maintenance、samples	作用域隔离、检索解释、维护、遗忘及论文语料示例生成
Loop 定义	GET/ POST/ PUT/ DELETE / api/ workflows	查询登录可用；写操作仅 ADMIN
Loop 运行	GET / runs; GET / runs/:runId; POST / stream; POST / workflow-runs/:runId/resume cancel	按用户和组织过滤；返回流、步骤状态、触发信息和恢复语义
调度管理	GET /api/scheduler/status; POST / api/workflows/:id/trigger	ADMIN 查看租约、队列和下次执行时间；手动触发复用统一 Run 管线
内部事件	POST /api/internal/events; POST / api/internal/events/:eventId/replay	服务身份认证；写入 Inbox、匹配触发器、幂等创建 Run
运行事件	GET / api/ workflow- runs/:runId/ events	返回状态转换、步骤、工具、审批、重试和调度事件

测内容使用明确的数据标记，并在进入 Prompt 前应用大小、类型和敏感字段策略；任何出现在数据中的“指令”都不改变系统层规则。模型输出只表达调用意图，实际执行由服务端 Schema、RBAC、工具注册表、资源所有权和策略引擎决定。这一设计直接覆盖 Prompt Injection、过度代理和不安全工具调用等 LLM 应用风险[12]。

输入安全引擎先执行 NFKC 归一化和零宽字符清理，再检测指令覆盖、系统提示词或凭证提取、安全与审批绕过、授权角色混淆、不可信工具输出提升为指令以及编码执行载荷。检测器输出 finding 列表和 0-100 风险分；意图分类器把请求归入 observe、diagnose、change、privilege-escalation、secret-access 或 unknown。策略引擎将两类结果合并为 allow、require-approval 或 deny。提权、凭证读取和高风险注入直接拒绝；变更请求进入审批；只读诊断继续执行。

工具侧采用第二道独立过滤。命令分析器解析引号、转义和 Token，拒绝分号、管道、重定向、命令替换、换行、Shell 解释器、sudo su、网络反弹程序和破坏性磁盘/账户操作；只有固定 scry-ops 包装器或 sudo -n 包装器形式可以进入审批。执行前策略同时校验主机用户名、toolId、commandId、命令哈希和策略结果，因此模型或自定义 Loop 不能绕过底层强制层。

统一审计表 scry_security_audit_events 使用 traceId 串联以下事件组：

- 输入与分类：intent.received、prompt_injection.s canned、intent.classified；

- 策略与审批：policy.*、approval.*；
- 工具执行：command.risk_evaluated、tool.execution.*。

事件只保存输入 SHA-256、结构化原因、风险分、目标和耗时，不保存 API Key、私钥、MCP 认证头或完整 Prompt。管理界面以时间线展示同一 Trace 的完整决策与执行链。

工具注册表为每个工具保存 riskLevel、requiredRole、approvalMode、timeout、resultLimit、allowedResources 和 policyVersion。调用前，Policy Evaluator 使用不可变请求上下文计算 allow、deny 或 require-approval。查询工具只接受结构化查询对象；SSH 工具只接受主机与命令 ID；模型不能传入数据库连接、文件路径、外部 URL 或原始 Shell。操作模式只改变可候选工具集合，不改变底层授权。

D. 秘密、密钥与网络控制

模型 API Key 和 SSH Secret 通过 AES-256-GCM 加密。每次加密生成 12 字节随机 IV，密文格式为

$$B = v \parallel keyId \parallel b64url(IV) \parallel b64url(Tag) \parallel b64url(C). \quad (17)$$

GCM 同时提供机密性和完整性，其底层 AES 标准由 NIST FIPS 197 定义[13]。根密钥由部署环境的 Secret Manager/KMS 注入，Agent 数据库只保存密文、keyId 和版本。密钥轮换采用双读单写：新写入使用当前 keyId，

读取可使用当前与上一版本，后台重加密完成后撤销旧版本。API 只返回 `hasApiKey` 或 `hasSecret`，日志脱敏器会移除 Token、Key、密码、私钥和带凭据 URL。

生产入口由反向代理终止 TLS，浏览器只访问同源 HTTPS。CORS 使用显式 Origin 列表，内部端口由网络策略隔离。Agent 对外模型端点使用协议与主机允许列表，远程 MCP 插件使用独立认证头；SSH 出口只允许登记主机与端口。ClickHouse、PostgreSQL、Agent SQLite、Query Internal API 和管理端点不直接暴露到公共网络。

E. 审计与追溯

审计覆盖登录主体、模型设置、工具策略、SSH 主机、命令定义、Loop 定义、调度触发、Run 状态、工具调用、审批和密钥轮换。每条审计事件包含 `eventId`、`orgId`、`actorType`、`actorId`、`action`、`resourceType`、`resourceId`、`result`、`reason`、`requestId`、`runId`、`traceId`、`sourceIp`、`userAgent` 和 `timestamp`。高敏感字段只保存哈希或摘要。

Run Event 与安全审计使用只追加写入。管理界面支持按用户、资源、动作、结果、时间和 `traceId` 查询；导出任务按时间窗口生成 JSON Lines，并包含完整性哈希。审计保留周期与遥测 TTL 独立配置，删除业务对象不会级联删除审计记录。SSH 输出正文不进入审计表，只保存字节数、内容哈希、截断标记和错误摘要；需要查看执行证据时从具有权限的原 Run 读取。

IX. 可靠性、性能与可运维性

A. 启动顺序与健康模型

服务依赖形成有向启动图：ZooKeeper → ClickHouse → Schema Migrator → Query Service → Agent/Alertmanager/Collector → Frontend。Schema Migrator 在 ClickHouse 就绪后执行结构升级，成功退出后 Query Service 才开始提供完整查询。Agent Service 启动时依次完成配置校验、数据库迁移、密钥可用性检查、Repository 初始化、调度租约注册和 HTTP 监听。

每个服务提供 `liveness` 和 `readiness`。`liveness` 只判断进程事件循环或主线程；`readiness` 同时检查关键依赖、迁移版本和任务接收能力。Agent `readiness` 检查 PostgreSQL、WASM SQLite、Query Service 身份端点和内置 MCP Server；外部模型供应商不作为启动前置条件，连接状态由模型测试与运行错误独立呈现。优雅下线时服务停止接收新请求，等待活动流和工具结束，再释放 MCP 与数据库连接。

B. 容量边界与资源预算

表 IX 给出默认边界。单请求上限与租户预算同时生效，取更严格值。预算器在模型调用前预留 Token 和费用，在流结束后按实际 usage 结算；预留失败会在模型调用之前返回 429，避免执行到一半才终止。

Query Service 对时间范围、步长、序列数、Limit 和查询超时实施独立保护。前端图表根据可视宽度计算目标点数，Agent 默认使用更窄时间窗口和聚合查询。ClickHouse 资源组区分交互查询、告警评估和后台任务，避免大规模仪表盘刷新阻塞告警与 Agent 取证。

C. 高可用与横向扩展

Frontend、Query Service、Collector 和 Agent Service 均采用无会话或外置状态设计，可在负载均衡器后横向扩展。浏览器流式请求使用连接亲和或支持长连接的网关；Mastra Memory 和运行数据保存在 PostgreSQL，Agent 业务配置与审计保存在可复制备份的 SQLite 文件中。多副本部署共享 PostgreSQL，并通过统一入口保持流连接稳定。

ClickHouse 使用分片与副本扩展写入和查询，ZooKeeper 保存协调元数据；Collector 按 OTLP 连接和队列水平扩展。Query Service 查询缓存使用共享缓存层或稳定的本地缓存失效策略。Alertmanager 以集群模式维护告警去重和通知状态。所有有状态组件部署反亲和规则与持久卷，避免单节点故障同时损坏主副本。

Run Worker 领取任务时写入 `workerId`、`leaseEpoch` 和 `leaseExpiresAt`，并周期续租。Worker 失联后，Recovery Worker 只回收租约过期且没有完成事件的 Run；已完成步骤依据检查点跳过。这个机制使模型调用或查询节点在实例退出后可恢复，同时避免活动 Worker 被第二个副本并发推进。

D. 自观测

Scry 使用 OpenTelemetry 观测自身。HTTP 请求、模型调用、Query Service 工具、Loop 节点、调度触发、事件消费、审批等待和 SSH 执行都创建 Span，并传播 `traceparent`。核心属性包括 `orgId` 的哈希、`userId` 的哈希、`threadId`、`runId`、`workflowId`、`stepId`、`toolId`、`provider`、`model`、`triggerType`、`status` 和 `error.type`；Prompt、秘密和完整工具结果不写入 Span。

Agent 暴露请求量、错误率、流持续时间、首 Token 延迟、模型 Token、估算费用、工具耗时、队列深度、Run 状态、步骤重试、审批等待、Cron 延迟、事件 Inbox Lag、幂等冲突、租约切换、SSH 结果和数据库事务耗时等指标。关键告警包括：readiness 连续失败、队列持续增长、调度延迟超过阈值、事件 Dead Letter 增长、模型错误率升高、配额拒绝突增、审批超时积压、SSH 指纹不匹配和密钥解密失败。

结构化日志使用 `requestId`、`traceId` 和 `runId` 关联。日志级别按操作结果划分：正常状态转换使用 `info`，可恢复依赖错误使用 `warn`，终态系统错误使用 `error`，调度扫描和状态细节使用 `debug`。日志输出经过统一脱敏，并限制对象深度和字符串长度。

E. 一致性与恢复

遥测写入采用最终一致性，Collector 批处理和 ClickHouse 物化视图会引入秒级可见延迟；Query Service 在响应元数据中返回实际时间范围和查询完成时间。平台配置、Loop 定义和运行状态采用事务一致性。Workflow 编辑使用版本号进行乐观锁，客户端提交旧版本时返回 409；Run 始终引用启动时的版本。

对话流使用客户端分支与持久化分支并行消费。客户端断开不会中止持久化分支，除非用户显式点击停止；停止操作写入 `cancelled` 事件并向模型传播 Abort Signal。Assistant Message 使用 Upsert，网络重连和重复完成回调不会产生两条消息。审批决策和 Trigger Claim 使用唯一键，重复请求返回现有结果。

TABLE VIII
安全控制与强制验证点

控制域	执行路径	审计结果
遥测访问	用户 JWT 透传 Query Service; RBAC 与 orgId 双重过滤; Agent 不持有 ClickHouse 凭据	记录用户、查询工具、时间窗口、结果规模和 traceId
Prompt 数据边界	系统/管理员/用户/附件/工具分层; 附件类型与大小校验; 敏感字段遮蔽	记录策略版本、输入摘要和阻断原因
工具授权	Zod Schema、角色、资源所有权、风险等级、审批模式和结果上限联合判定	记录 toolCallId、参数摘要、决策和耗时
SSH 执行	指纹固定、固定命令 ID、无 PTY、无交互、超时、输出上限、逐次审批	记录主机、命令、策略版本、审批人和退出状态
秘密保护	AES-256-GCM、KMS Key ID、双读单写轮换、响应与日志脱敏	记录创建、使用、轮换和撤销事件, 不记录明文
事件入口	服务身份、请求签名、时间窗口、eventId 幂等和载荷 Schema	记录 source、eventType、匹配结果和 Run 关联
资源治理	用户/组织/Loop 并发、速率、Token、费用和输出预算	记录配额维度、当前值、阈值和拒绝码

X. 开发、构建与生产部署

A. 开发环境

开发环境通过 Makefile.dev 封装 Compose, 保留与服务相同的服务边界。常用命令如下:

```
make -f Makefile.dev dev
make -f Makefile.dev dev-ps
make -f Makefile.dev dev-logs
make -f Makefile.dev dev-backend-restart
make -f Makefile.dev dev-agent-restart
make -f Makefile.dev dev-frontend-restart
make -f Makefile.dev dev-down
```

前端使用 Webpack Watch, Agent 使用 tsx watch, Go 服务通过重启触发编译。环境文件只保存本地非生产配置, 模型 Key 和 SSH Secret 通过管理界面写入加密存储。开发入口绑定回环地址, 数据卷在 dev-down 后保留, 清理卷使用单独的显式命令。Agent 启动前由一次性 agent-data-init 任务修正持久卷属主; 该任务以只读根文件系统运行, 删除全部 Linux Capability 后仅恢复 CHOWN, 完成即退出。Agent 主容器始终使用非 root 用户、cap_drop=ALL 与 no-new-privileges, 卷升级不需要放宽主进程权限。

B. 端口与网络分区

默认服务端口为: Web Console 3301、Query Public API 8080、Query Internal API 8085、Agent API 4111、ClickHouse Native/HTTP 9000/8123、Alertmanager 9093、OTLP gRPC/HTTP 4317/4318。生产环境只公开 HTTPS 443 和按需公开 OTLP 接入端口; 其他端口位于应用、数据和管理三个网络分区。

应用分区包含 Frontend、Query Service 和 Agent Service; 数据分区包含 ClickHouse、ZooKeeper、PostgreSQL 和 Agent SQLite 备份; 管理分区包含内部 API、迁移任务和备份任务。网络策略允许 Collector 写 ClickHouse、Query Service 读 ClickHouse、Agent 调 Query Service、MCP 与模型端点、Alertmanager 回调内部 API。浏览器不能直接访问数据分区, Agent 不能直接访问 ClickHouse。

C. 生产部署流程

生产发布使用版本化镜像和不可变配置。部署顺序为: 创建命名空间与网络策略, 注入数据库和密钥 Secret, 部署 ZooKeeper/ClickHouse, 执行 Schema Migrator, 部署 Query Service 与 Alertmanager, 部署 Agent Service, 部署 Collector, 最后发布 Frontend 与入口网关。每一阶段都等待 readiness, 再进入下一阶段。

Agent Service 启动前必须提供主密钥 Key ID、数据库 URL、Query Service URL、允许的模型端点和内部服务令牌。模型业务配置由管理员在系统内管理, 不写入镜像。SSH 主机通过管理页面逐台纳管并完成指纹核验。Loop 在草稿状态完成保存和试运行, 启用后调度器才注册 Cron 与事件触发器。

滚动发布时, 新副本完成迁移兼容性检查并进入 readiness, 旧副本撤销 readiness 后停止领取 Run。调度租约自动转移, 活动 Run 由旧副本完成或保存检查点后交由新副本恢复。入口网关对 UI Message Stream 设置足够长的空闲超时, 并关闭响应缓冲。

D. 麒麟 V11 与 LoongArch64 原生部署

deploy/kylin-loong64 提供完整的宿主机原生部署链。build-node.sh 下载 Node.js 22 源码与官方 SHA-256 清单, 在目标机编译后验证 process.arch=loong64; build-query.sh 使用目标机 GOARCH=loong64 和 CGO_ENABLED=1 构建 Query Service, 并通过 file 检查二进制架构。Agent 先用 TypeScript 生成 dist, 运行阶段只安装生产依赖, 不使用 tsx/esbuild。

Agent 业务数据库由 sql.js WebAssembly 驱动, 使用与 SQLite 兼容的文件格式、串行事务和临时文件原子重命名落盘; Mastra Memory 和运行存储使用麒麟原生 PostgreSQL。该组合不加载仅提供 x64/arm64 预编译包的 Node 原生 SQLite 扩展。Frontend 构建为静态资源, 由 Nginx 同源代理 Query Service 和 Agent Service。systemd 单元统一使用无登录的 scry 用户, 启用 NoNewPrivileges、ProtectSystem=strict、ProtectHome 和受限写目录。

部署流程由 preflight.sh、setup-postgres.sh、构建脚本、install-native.sh 与 acceptance.sh 组成。预

TABLE IX
Agent、Loop 与租户资源边界

对象	默认值/范围	执行方式
模型输出 Token	默认 8,192; 范围 64 – 128,000	供应商参数与组织预算共同约束
模型步骤与重试 模型总体超时	8 步; 1 次供应商重试 120 秒; 范围 10 – 1,800 秒	达到上限后生成明确终态, 不重复副作用工具 Abort Signal 传播到模型流和只读工具
记忆窗口	默认 30; 范围 1 – 200 条	按 userId/threadId 读取最近消息
TopoMem 候选	最多扫描 800 条; 默认返回 5 条	按组织和作用域过滤后执行路径与分块评分
TopoMem 路径	错误率阈值 0.10; 最多 3 条; 最多 6 个服务节点	当前证据不足时显式进入 lexical-fallback
记忆正文与分块	单条 120,000 字节; 单块 1,800 字符; 最多 80 块	稳定序列化、秘密遮蔽、哈希去重和类型标注
工具结果 消息附件	200,000 字节 5 个; 单文件 10 MB; 文本 512 KB	保留结构、摘要、样本和截断标记 MIME、扩展名、大小和文件名校验
Loop 结构	100 节点; 5 层; 6 并行分支	保存和编译阶段双重校验
循环与 foreach SSH	50 次/项; 并发 1 – 10 5 – 120 秒; 输出 200 KB	每次迭代写入 itemKey 与步骤事件 禁止交互; 超时主动关闭连接
用户交互 Run	默认每用户 3 个活动 Run	超额请求进入队列或返回配额状态
组织 Run API 速率	默认 20 个活动 Run 用户 60 req/min; 内部事件 600 req/min	Dispatcher 公平调度并按优先级出队 令牌桶; 返回 Retry-After
Token/费用预算	按日和按月配置	预留、结算、告警和硬停止四阶段

检验证麒麟 V11、LoongArch64、Go/Node/GCC/PostgreSQL/Nginx, 并通过原生客户端对 ClickHouse 执行 SELECT 1。验收脚本检查 systemd 状态、HTTP 健康接口、服务用户、数据目录属主和可选 MCP 工具目录, 从而把平台适配从说明性声明变为可重复执行的交付步骤。

E. 备份与恢复

备份分为遥测数据、平台配置和智能执行状态三个集合。ClickHouse 使用分区备份或对象存储备份; Query Service SQLite/数据库使用一致性备份; Agent 数据库在事务边界生成备份并单独保存。KMS 根密钥不进入数据库备份, Key ID、密钥版本和恢复权限通过 Secret Manager 的灾备机制保存。

默认策略为每日全量、每 15 分钟增量或日志归档, 保留 7 个日版本、4 个周版本和 12 个月版本。生产目标为配置与 Agent 状态 RPO 15 分钟、RTO 60 分钟; 遥测数据的 RPO/RTO 根据保留量和 ClickHouse 规模单独计算。备份任务完成后执行可读性校验、对象计数、校验和与密钥版本检查。

恢复顺序为: 恢复密钥访问, 恢复 ZooKeeper/ClickHouse 与模式, 恢复 Query Service 配置库, 恢复 Agent 库, 执行迁移检查, 启动 Query Service, 启动 Agent, 最后启动 Collector 和 Frontend。Agent 恢复后, Recovery Worker 扫描活动 Run; 超过审批有效期的 suspended Run 标记 expired, 租约过期的 running Run 重新入队, 已完成

Run 保持终态。每次恢复演练记录恢复点、耗时和对象数量。

F. 升级与数据库迁移

所有服务版本遵循向前兼容的滚动升级顺序。数据库迁移采用 expand-and-contract: 先添加新表、新列和索引, 使新旧版本都可读写; 完成应用滚动升级和数据回填后, 再在后续版本移除旧结构。迁移表记录 version、checksum、startedAt、completedAt 和 executorId, 多副本通过迁移锁保证只执行一次。

Workflow DSL 和 A2UI 都携带版本号。编译器保留对已发布 DSL 版本的读取能力, 保存时可将旧定义转换为当前版本并生成新的 Workflow Version; 运行中的 Run 继续使用原版本。A2UI 前端对已知版本进行专用渲染, 对更高版本保留 JSON 回退。模型供应商适配层以协议能力检测控制参数, 升级模型不会改变已开始 Run 的配置副本。

G. 运维 Runbook

日常运维以健康、队列、依赖、配额和运行状态为主线。

- 1) **Agent 无法访问:** 检查入口 5xx、Agent readiness、PostgreSQL、WASM SQLite 和 Query Service 身份端点; 使用 requestId/traceId 定位认证、MCP 或依赖错误。

- 2) **模型响应异常**: 在 AI 设置执行连接测试, 核对 provider、model、endpoint、超时、限流和组织预算; 供应商错误通过 category 分类。
- 3) **Loop 长时间排队**: 检查实例/组织/Loop 并发、Dispatcher 队列深度、Worker 租约和 nextAttemptAt; 扩容 Worker 后队列自动消化。
- 4) **Cron 没有按时运行**: 检查 Loop enabled、Cron、timezone、Scheduler Leader、nextFireAt、misfire 记录和 Trigger Claim; 按 triggerKey 判断是否被幂等抑制。
- 5) **事件没有触发**: 按 eventId 查询 Inbox, 检查签名、eventType、filter、orgId、matched 状态和 Dead Letter; 修复后使用 replay 端点重投。
- 6) **审批无法恢复**: 检查 Run 是否 suspended、审批是否到期、当前用户角色、suspendedStep 和 workflowVersion; 409 表示状态已被其他决策推进。
- 7) **SSH 连接失败**: 依次检查主机启用状态、网络、指纹、用户名、凭据、策略和超时; 指纹变化必须重新走可信核验。
- 8) **查询结果过大**: 缩小时间范围、增加步长、减少 Group By 和 Limit; 工具返回中的 truncated 与 originalBytes 用于确认截断。
- 9) **恢复后 Run 异常**: 查询 Run Event 的最后序号和 worker lease, 确认 Recovery Worker 是否重新入队; 副作用步骤依据 executionKey 核查审计后再处理。

XI. 兼容性与扩展机制

A. 模型与工具扩展

模型适配器以统一的 generate/stream/tool-call 接口隔离 OpenAI Responses 与 Anthropic Messages。新增供应商时实现模型工厂、Endpoint 规范化、错误分类、连接测试和 usage 映射, 不改变 Agent、Thread、A2UI 或 Loop。模型设置保存 provider、protocol、model 和参数副本, 运行事件记录实际使用值。

新增工具需要声明唯一 toolId、输入/输出 Schema、风险等级、角色、审批模式、超时、结果上限、A2UI 映射和审计字段。只读工具通过当前用户 JWT 访问领域服务; 有副作用工具实现 executionKey 和明确终态。工具注册完成后即可被普通 Agent 和 Loop Agent 节点复用, 前端对标准 A2UI 类型无需新增页面逻辑。

B. Loop 与事件扩展

Loop DSL 通过 version 和节点 type 扩展。新节点实现验证器、编译器、运行事件映射、编辑器配置和迁移函数; 旧版本定义保持可执行。事件系统使用开放的 eventType 命名空间, 新增事件源只需生成标准信封并配置服务身份。过滤器只读取信封, 不与具体来源代码耦合。

Webhook、工单、变更平台和配置中心等外部系统通过 Integration Adapter 转换为内部事件。出站动作通过受控工具实现, 继续复用角色、审批、幂等、预算和审计, 不在 Loop 编译器中直接写第三方 API 逻辑。

XII. 结语

Scry 将遥测接入、分析查询、平台管理和智能执行组织为边界清晰的四平面架构。指标、日志与链路由 OpenTelemetry 和 ClickHouse 提供统一事实基础; Query

Service 集中承担查询语义、RBAC、告警、仪表盘和配置; Agent Service 则以 TopoMem、证据链、确定性 MCP 和安全 Loop 形成系统的技术主线。四者分别解决“先查哪里”“依据什么”“如何可靠调用”和“怎样重复执行”, 使智能诊断不脱离可观测平台的真实数据、权限和运行状态。

TopoMem 使用本次事故 Trace 导航到异常调用链下游, 以错误签名和记忆分块区分同一服务的不同故障模式, 并在 Trace 缺失、终端无记忆或多路径无支持时显式回退。它与证据系统共同构成验证后学习闭环: 历史记忆只生成假设, 本轮工具结果形成证据, 人工接受或 Loop 验证后的结果才能写回, 维护器再执行衰减、强化、合并、冲突挂起和归档。MCP 与 Loop 位于闭环的执行侧, 前者提供严格参数与权限边界, 后者提供版本、审批、检查点、幂等和恢复语义。由此, Scry 把一次自然语言诊断转化为可解释、可治理、可积累并可重复运行的技术过程。

Appendix A

Agent API 参考

TABLE X
Agent Service HTTP API

方法	路径	权限	语义
GET	/health	Public	进程、版本、readiness 摘要
GET	/api/settings/model	ADMIN	返回模型配置和秘密存在状态
PUT	/api/settings/model	ADMIN	更新供应商、端点、模型、参数和记忆窗口
POST	/api/settings/model/test	ADMIN	使用表单与已保存秘密合并值执行连接测试
POST	/mcp	Login	MCP Streamable HTTP initialize、tools/list 与 tools/call
GET	/api/mcp/tools	Login	返回当前用户可见的内置 MCP 工具目录
GET	/api/mcp/servers	ADMIN	按组织列出远程 MCP 插件，不返回认证头明文
POST	/api/mcp/servers	ADMIN	注册远程 Streamable HTTP MCP 服务
PUT	/api/mcp/servers/:serverId	ADMIN	更新插件、超时、启用状态和加密请求头
DELETE	/api/mcp/servers/:serverId	ADMIN	删除组织内 MCP 插件
POST	/api/mcp/servers/:serverId/test	ADMIN	建立 MCP 连接并列出工具
GET	/api/ssh/hosts	ADMIN	列出组织内 SSH 主机，不返回秘密
POST	/api/ssh/hosts/probe	ADMIN	读取远端主机公钥指纹
POST	/api/ssh/hosts	ADMIN	创建主机并加密保存认证材料
GET	/api/ssh/hosts/:hostId	ADMIN	获取主机配置和秘密存在状态
PUT	/api/ssh/hosts/:hostId	ADMIN	更新主机、指纹、凭据和启用状态
DELETE	/api/ssh/hosts/:hostId	ADMIN	删除主机与策略引用
POST	/api/ssh/hosts/:hostId/test	ADMIN	执行指纹与认证握手测试
GET	/api/ssh/policies	ADMIN	返回只读与扩展执行策略
PUT	/api/ssh/policies/:toolId	ADMIN	更新启用、审批、超时和命令定义
GET	/api/ssh/audits	ADMIN	按用户、主机、工具、结果和时间查询审计
GET	/api/security/audits	ADMIN	按 traceId 查询输入、策略、审批和执行事件
GET	/api/threads	Login	按当前用户列出 Thread
POST	/api/threads	Login	创建 Thread
GET	/api/threads/:threadId	Owner	返回 Thread 与完整 UI Message
DELETE	/api/threads/:threadId	Owner	删除 Thread、消息和关联运行
POST	/api/chat	Owner	普通 Agent 流式请求、重新生成或恢复
GET	/api/threads/:threadId/evidence	Owner	返回证据并从历史工具 Part 补建缺失证据
PATCH	/api/evidence/:evidenceId	Owner	更新接受、驳回、挂起、选择状态或备注
POST	/api/threads/:threadId/evidence/revise	Owner	使用选定证据生成回答修订并记录排除项
GET	/api/memories/stats	Login	返回总量、在线量、验证量、挂起量、平均活性和分布
GET	/api/memories	Login	按查询、类型、作用域、状态和服务过滤记忆
POST	/api/memories	Login/Admin	创建记忆：组织级写入要求 ADMIN；可绑定证据 UUID
GET	/api/memories/:memoryId	Visible	返回正文、分块、来源、关系和维护历史
PATCH	/api/memories/:memoryId	Visible/Admin	更新状态、固定、备注、置信度和重要性
DELETE	/api/memories/:memoryId	Visible/Admin	清空可检索内容并保留遗忘审计骨架
POST	/api/memories/search	Login	执行 TopoMem/TopoMem-MP/lexical-fallback 并返回评分解释
GET	/api/threads/:threadId/memories	Owner	使用当前 Thread 问题与证据生成记忆包
POST	/api/memories/maintenance	Login	执行衰减、强化、合并、冲突、归档和过期处理
GET	/api/memory-samples	Login	返回按类别组织的论文语料示例及生成状态
POST	/api/memory-samples/generate	Login	选择示例并生成成为正常参与检索的记忆
DELETE	/api/memory-samples/:memoryId	Login	删除指定示例记忆及分块、来源、关系和操作记录
GET	/api/workflows	Login	列出有权查看的 Loop
POST	/api/workflows	ADMIN	创建并验证 Loop 定义
GET	/api/workflows/:workflowId	Login	返回定义、输入、触发器、版本和下次执行
PUT	/api/workflows/:workflowId	ADMIN	乐观锁更新并生成版本记录
DELETE	/api/workflows/:workflowId	ADMIN	停用调度并删除定义
GET	/api/workflows/:workflowId/runs	Login	查询当前主体可见的运行历史
POST	/api/workflows/:workflowId/stream	Login	手动创建 Run 并返回 UI Message Stream
POST	/api/workflows/:workflowId/trigger	ADMIN	通过统一触发管线创建手动 Run
GET	/api/workflow-runs/:runId	Owner/Admin	返回 Run、步骤、触发器、统计和输出
GET	/api/workflow-runs/:runId/events	Owner/Admin	返回有序 Run Event
POST	/api/workflow-runs/:runId/resume	Approver	对 suspended Run 提交批准或拒绝
POST	/api/workflow-runs/:runId/cancel	Owner/Admin	取消 queued/running/retrying/suspended Run
GET	/api/scheduler/status	ADMIN	返回 Leader、租约、队列、延迟和任务注册
POST	/api/internal/events	Service	验证并写入事件信封
POST	/api/internal/events/:eventId/replay	ADMIN	重新投递 Dead Letter 或历史事件

Appendix B Agent 数据模型

TABLE XI
Agent 关键表与字段

表	关键字段	职责与约束
scry_model_settings	orgId, environment, provider, endpointMode, model, baseUrl, apiKeyEncrypted, keyVersion, generationOptions, updatedAt	组织级模型配置；唯一键为 orgId+environment；API 只返回秘密存在状态
scry_agent_threads	id, orgId, userId, title, createdAt, updatedAt	Thread 所有权与列表排序
scry_agent_ui_messages	id, threadId, role, messageJson, createdAt, updatedAt	保存完整 UI Message Part；messageId Upsert
scry_evidence_items	id, threadId, orgId, userId, traceId, messageId, sourceType, sourceName, contentJson, confidence, relevance, status, selected, note, capturedAt	工具与文档证据主体；保存评分、生命周期和用户选择
scry_evidence_relations/actions/revisions	evidenceId, relationType, score, action, previousStatus, nextStatus, selectedIdsJson, excludedIdsJson, instruction, createdAt	证据关系、人工处置和基于更新证据的回答修订
scry_memory_items	id, orgId, userId, threadId, workflowId, memoryType, scope, serviceName, contentHash, confidence, importance, vitality, status, pinned, expiresAt	TopoMem 主记录；按作用域、服务、状态和活性检索
scry_memory_chunks	id, memoryId, chunkIndex, chunkType, contentText, contentHash, tokenCount, keywordsJson, serviceName, confidence	稳定分块、预览和块级检索；memoryId+chunkIndex 唯一
scry_memory_sources/rerelations/actions	memoryId, sourceType, sourceId, traceId, relation, weight, relationType, score, action, status, detail	记忆来源、支持/冲突/重复关系及维护历史
scry_memory_topology_edges	orgId, fromService, toService, observationCount, errorObservationCount, cumulativeErrorRate, confidence, lastTraceId, lastObservedAt	运行拓扑观察累计；不替代当前事故 Trace
scry_memory_retrieval_runs	id, orgId, userId, threadId, traceId, queryText, strategy, symptomService, signatureJson, pathsJson, fallbackReason, candidateCount, returnedIdsJson, latencyMs	TopoMem 检索决策审计和可解释复盘
scry_mcp_servers	id, orgId, name, url, enabled, headersEncrypted, timeoutMs, createdAt, updatedAt	远程 MCP 插件注册；认证头只以 AES-GCM 密文保存
scry_ssh_hosts	id, orgId, name, hostname, port, username, authType, secretEncrypted, keyVersion, fingerprint, enabled	SSH 主机、认证密文和固定指纹
scry_tool_policies	orgId, toolId, riskLevel, enabled, requireApproval, policyJson, version, updatedAt	工具启用、命令定义、超时和审批版本
scry_ssh_audit_logs	id, orgId, userId, runId, toolCallId, hostId, commandId, policyVersion, status, exitCode, durationMs, stdoutBytes, stderrBytes, outputHash, createdAt	SSH 成功、失败和拒绝审计
scry_security_audit_events	id, traceId, orgId, userId, eventType, source, target, decision, riskScore, inputHash, detailsJson, createdAt	输入扫描、意图分类、策略、审批和工具执行统一审计
scry_workflows	id, orgId, ownerUserId, name, description, enabled, definitionJson, tagsJson, scheduleJson, eventTriggersJson, inputSchemaJson, version, concurrencyPolicy, executionPrincipal, createdAt, updatedAt	Loop 当前版本、触发器、身份和并发策略
scry_workflow_versions	workflowId, version, definitionJson, checksum, createdBy, createdAt	不可变历史版本；workflowId+version 唯一
scry_workflow_runs	id, orgId, userId, workflowId, workflowVersion, threadId, triggerType, triggerKey, status, attempt, inputJson, outputJson, stepStateJson, suspendedStep, scheduledAt, nextAttemptAt, workerId, leaseEpoch, createdAt, updatedAt	Run 当前状态副本；状态更新使用 expectedStatus
scry_workflow_run_events	runId, sequence, eventType, actor, payloadSummary, traceId, createdAt	只追加状态事件；runId+sequence 唯一
scry_trigger_claims	triggerKey, orgId, workflowId, triggerType, sourceId, scheduledAt, runId, createdAt	Cron/Event 筹等；triggerKey 唯一
scry_event_inbox	eventId, replayId, orgId, eventType, source, envelopeJson, status, attempts, nextAttemptAt, errorCode, receivedAt, processedAt	事件持久接收、匹配、重试与 Dead Letter
scry_runtime_leases	name, holderId, epoch, expiresAt, updatedAt	Scheduler Leader 与 Worker Run Lease
scry_agent_audit_events	eventId, orgId, actorType, actorId, action, resourceType, resourceId, result, requestId, runId, traceId, metadataJson, createdAt	Agent 管理和执行统一审计

Appendix C
Agent 配置环境变量

TABLE XII
Agent Service 环境变量

变量	默认值	含义
SCRY_AGENT_PORT	4111	HTTP 监听端口
SCRY_AGENT_HOST	0.0.0.0	容器内监听地址
SCRY_QUERY_SERVICE_URL	必填	Query Service Public API 地址
SCRY_QUERY_INTERNAL_URL	必填	身份换取与内部事件协作地址
SCRY_AGENT_DATABASE_URL	file:/ var/ lib/ scry/ agent/agent.db	WASM SQLite 业务数据库文件
SCRY_AGENT_POSTGRES_URL	必填	Mastra Memory 与运行存储的 PostgreSQL URL
SCRY_MCP_INTERNAL_URL	http:// 127.0.0.1:4111/ mcp	内置 MCP Server 地址
SCRY_MCP_TIMEOUT_MS	30000	MCP 初始化、工具目录和调用超时
SCRY_AGENT_MASTER_KEY	必填	本地 Secret 模式下的高熵主密钥
SCRY_AGENT_KMS_KEY_ID	生产必填	KMS 当前加密 Key ID
SCRY_AGENT_PREVIOUS_KMS_KEY_ID	空	轮换期间允许读取的上一 Key ID
SCRY_AGENT_MAX_TOOL_RESULT_BYTES	200000	单次工具结果上限
SCRY_AGENT_MAX_ACTIVE_RUNS	100	实例活动 Run 上限
SCRY_AGENT_MAX_ORG_RUNS	20	单组织活动 Run 上限
SCRY_AGENT_MAX_USER_RUNS	3	单用户交互 Run 上限
SCRY_AGENT_RUN_LEASE_SECONDS	30	Worker Run Lease 时长
SCRY_AGENT_SCHEDULER_LEASE_SECONDS	15	Scheduler Leader Lease 时长
SCRY_AGENT_SCHEDULER_SCAN_SECONDS	5	Cron 到期扫描周期
SCRY_AGENT_EVENT_BATCH_SIZE	100	Event Inbox 单批消费数量
SCRY_AGENT_EVENT_MAX_ATTEMPTS	8	事件消费最大尝试次数
SCRY_AGENT_EVENT_SIGNATURE_SECRET	必填	内部事件 HMAC 服务密钥
SCRY_AGENT_ALLOWED_MODEL_HOSTS	官方域名	自定义模型端点允许列表
SCRY_AGENT_CORS_ORIGINS	同源	允许访问 Agent API 的 Origin 列表
SCRY_AGENT_OTEL_ENDPOINT	Collector 地址	OTLP Span、Metric 和 Log 导出端点
SCRY_AGENT_LOG_LEVEL	info	结构化日志级别
SCRY_AGENT_SHUTDOWN_GRACE_SECONDS	30	优雅下线等待时间

Appendix D

Loop 定义示例

下面的定义表示：读取服务与告警，并行生成诊断和仪表盘上下文；当存在严重告警时进入人工审批，批准后执行 Agent 处置步骤。该 Loop 每 15 分钟运行，也响应 alert.firing 事件。

```
{
  "version": 2,
  "name": "production-health-diagnosis",
  "enabled": true,
  "tags": ["production", "health"],
  "schedule": {
    "enabled": true,
    "cron": "* / 15 * * * *",
    "timezone": "Asia/Shanghai",
    "misfirePolicy": "fire-once"
  },
  "eventTriggers": [
    {
      "id": "critical-alert",
      "eventType": "alert.firing",
      "filter": "attributes.severity in ['critical','high']",
      "enabled": true
    }
  ],
  "inputSchema": [
    {
      "id": "environment",
      "label": "environment",
      "type": "text",
      "required": true,
      "defaultValue": "production"
    }
  ],
  "concurrencyPolicy": "forbid",
  "definition": {
    "version": 2,
    "steps": [
      { "id": "services", "name": "list-services", "type": "services" },
      { "id": "alerts", "name": "list-alerts", "type": "alerts" },
      {
        "id": "parallel-context",
        "name": "collect-context-in-parallel",
        "type": "parallel",
        "parallel": {
          "branches": [
            {
              "id": "diagnosis",
              "name": "diagnosis",
              "steps": [{
                "id": "analyze",
                "name": "analyze-root-cause",
                "type": "agent",
                "prompt": "Locate the root cause from services and alerts."
              }]
            },
            {
              "id": "dashboards",
              "name": "dashboards",
              "steps": [{
                "id": "dashboard-list",
                "name": "list-dashboards",
                "type": "dashboards"
              }]
            }
          ]
        }
      }
    ],
    {
      "id": "critical-check",
      "name": "critical-alert-check",
      "type": "condition",
      "condition": {
        "rule": {
          "path": "results.alerts.criticalCount",
          "operator": "gt",
          "value": "0"
        }
      },
      "whenTrue": [
        {
          "id": "approve",
          "name": "approve-operation",

```

```
        "type": "approval",
        "approvalMessage": "Approve the production operation"
    },
    {
        "id": "operate",
        "name": "execute-operation",
        "type": "agent",
        "prompt": "Execute the approved operation and verify recovery."
    }
],
"whenFalse": []
}
}
}
```

Appendix E
运行状态与错误分类

TABLE XIII
Run 状态定义

状态	允许来源	语义与后续状态
queued	Trigger Claim 成功	等待 Worker；进入 running 或 cancelled
running	Worker 领取 queued/retrying	执行步骤：进入 suspended、retrying、completed、failed 或 cancelled
suspended	approval/tool approval	保存检查点等待决策；进入 queued、failed、expired 或 cancelled
retrying	可重试依赖或供应商错误	保存 nextAttemptAt；到期进入 queued，超过次数进入 failed
completed	所有步骤成功	终态；保存输出、usage、费用和完成事件
failed	终态错误或审批拒绝	终态；保存 error code、message、traceId 和步骤状态
cancelled	用户、管理员或 replace 策略	终态；传播 Abort Signal 并记录取消主体
expired	审批或运行有效期到期	终态；保留恢复点与到期原因

TABLE XIV
API 与运行错误分类

错误码	HTTP	Run 处理	含义
VALIDATION_ERROR	400	failed	Schema、Cron、过滤器、DSL 或参数错误
AUTHENTICATION_REQUIRED	401	failed	Token 缺失、过期或签名无效
FORBIDDEN	403	failed	角色、组织、所有权或工具策略拒绝
NOT_FOUND	404	failed	Thread、Workflow、Run、主机或命令不存在
STATE_CONFLICT	409	保持当前状态	版本、审批、幂等或状态转换冲突
QUOTA_EXCEEDED	429	queued/failed	速率、并发、Token 或费用预算达到阈值
PROVIDER_RATE_LIMIT	429	retrying	模型供应商限流
DEPENDENCY_UNAVAILABLE	503	retrying	Query Service、数据库、模型或 SSH 依赖暂时不可用
DEPENDENCY_TIMEOUT	504	retrying	外部依赖超过超时
APPROVAL_REJECTED	409	failed	审批人拒绝执行
RUN_CANCELLED	409	cancelled	用户、管理员或并发策略取消
INTERNAL_ERROR	500	failed/retrying	平台内部异常，响应携带 traceId

References

- [1] R. Zhang, H. Yang, and H. Liu, “Topomem: Runtime-topology-guided retrieval in the aiops agent memory loop,” 2026, technical manuscript.
- [2] OpenTelemetry Authors, “Opentelemetry specification,” OpenTelemetry documentation, 2026, available: <https://opentelemetry.io/docs/specs/otel/>. Accessed: Jul. 19, 2026.
- [3] ClickHouse, Inc., “What is clickhouse?” ClickHouse documentation, 2026, available: <https://clickhouse.com/docs/intro>. Accessed: Jul. 19, 2026.
- [4] Mastra, “Agents overview,” Mastra documentation, 2026, available: <https://mastra.ai/docs/agents/overview>. Accessed: Jul. 19, 2026.
- [5] —, “Workflows overview,” Mastra documentation, 2026, available: <https://mastra.ai/docs/workflows/overview>. Accessed: Jul. 19, 2026.
- [6] Vercel, “Ai sdk core: Tool calling,” AI SDK documentation, 2026, available: <https://ai-sdk.dev/docs/ai-sdk-core/tools-and-tool-calling>. Accessed: Jul. 19, 2026.
- [7] Hono Authors, “Hono documentation,” Hono documentation, 2026, available: <https://hono.dev/docs/>. Accessed: Jul. 19, 2026.
- [8] OpenAI, “Responses api reference,” OpenAI API documentation, 2026, available: <https://platform.openai.com/docs/api-reference/responses>. Accessed: Jul. 19, 2026.
- [9] Anthropic, “Messages api,” Anthropic API documentation, 2026, available: <https://platform.claude.com/docs/en/api/messages>. Accessed: Jul. 19, 2026.
- [10] M. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” Internet Engineering Task Force, Tech. Rep. RFC 7519, 2015.
- [11] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” National Institute of Standards and Technology, Tech. Rep. NIST SP 800-207, 2020.
- [12] OWASP Foundation, “Owasp top 10 for large language model applications,” OWASP project documentation, 2025, available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>. Accessed: Jul. 19, 2026.
- [13] National Institute of Standards and Technology, “Advanced encryption standard (aes),” U.S. Department of Commerce, Tech. Rep. FIPS 197-upd1, 2023.